JIMMA UNIVERSITY

JIMMA INSTITUTE OF TECHNOLOGY FACULTY OF COMPUTING AND INFORMATICS

Department of Computer Science

FINAL YEAR PROJECT - II (FYPr4212)

የጥናት ጎጆ | StudyNest

Name	ID
1. Abdurahman Seid	RU0234/14
2. Changkuoth Top	RU0348/14
3. Hailemichel Eshetu	RU1090/14
4. Yusra Reshid	RU0048/14
5. Tsion Argaw	RU1610/14

Advisor's name 1. Mrs. Tsedenya _____

2. Mr. Mizanu _____

A Final Year Project

Submitted to Faculty of Computing and Informatics, Jimma Institute of Technology, Jimma University, in Partial fulfillment of the requirement of the Degree of Bachelor of Science in Computer Science

Jimma, Ethiopia
May 2025

Acknowledgment

We would like to acknowledge the individuals who have provided their guidance and support during this project. Their input and assistance have been instrumental in ensuring the clarity and quality of this documentation.

We extend sincere appreciation to Mrs. Tsedenya and Mr. Mizanu, our esteemed advisors, for their invaluable guidance and support throughout the entire duration of this project. Their extensive expertise and insightful feedback have played a pivotal role in ensuring the coherence and accuracy of our project. Mr. Mizanu's depth of knowledge and Mrs. Tsedenya’s perspectives have significantly contributed to the project's overall quality.

Their dedication to providing constructive suggestions and thoughtful insights has not only refined the content but has also enhanced the project's robustness. We are truly fortunate to have had their mentorship, and their commitment to excellence has been a driving force behind our success. We would also like to thank our colleagues and friends for their support and encouragement. Their input and discussions have helped in the content and structure our project.

Additionally, we would like to acknowledge the contributions of all the individuals who have provided their time and expertise for reviewing and proofreading the proposal. Their attention to detail and constructive criticism have been immensely helpful in improving its overall quality

Table of Contents

Acknowledgment ii

List of Tables iv

List of Figures 5

Chapter One: Introduction 1

1.1 Background Information of the Organization 2

1.1.1 Vision of the Organization 2

1.1.2 Mission of the Organization 2

1.2 Background of the Project 3

1.3 Motivation 3

1.4 Statement of the problem 3

1.5 Objectives of the Project 4

1.5.1 General Objective 4

1.5.2 Specific Objectives 4

1.6 Feasibility Analysis 5

1.6.1 Operational Feasibility 5

1.6.2. Technical Feasibility 5

1.6.3. Economic Feasibility 6

1.6.4. Behavioral Feasibility 6

1.6.5. Schedule Feasibility 6

1.6.5. Economic Feasibility 6

1.7 Scope of the Project 9

1.8 Significance of the project 9

1.9 Target beneficiaries of the system 10

1.10. Methodology 11

1.10.1 Fact-Finding Techniques 13

1.10.2 Systems Analysis and Design 13

1.10.3. Development Tools 15

1.11. Limitation of the project 15

1.12 Team composition 16

Chapter Two: Requirement Analysis and Specification 17

2.1 Current System 17

2.2 Business Rules 17

2.3 Proposed System 18

2.3.1 Overview 18

2.4 Requirements of the Proposed System 18

2.4.1 Functional Requirements 18

2.4.2 Non-Functional Requirements 19

2.5 System Analysis 20

2.5.1 Use case model diagrams 20

2.5.2 Sequence diagram 22

2.5.3 State chart diagram 26

2.5.4 Activity diagram 26

2.5.5 Class diagram 30

Chapter Three: System Design 31

3.1 Introduction 31

3.2 Purpose of the system 31

3.3 Design Goals 32

3.4 Deployment Diagram 33

3.5 Architecture design 33

3.5.1. Subsystem decomposition 34

3.5.2. Component diagram 37

3.5.2. Persistent modeling 38

3.5.3. Access control and security 39

3.5.4. Global software control 39

3.5.5. Boundary conditions 40
3.6. User Interface Design 40
3.7 Database Design 42
Chapter Four: Implementation and Testing 46
4.1 Introduction 46
4.2 Overview of the Programming Language Used 46
4.3 Sample User Interfaces 48
4.4 Application Testing 51
4.4.1 Unit Testing 51
4.4.3 Testing Objectives 53
4.4.4 Test Procedures 53
4.5 Integration Testing 54
4.6 System Testing 54
4.7 User manual preparation 55
4.7.1 Introduction 55
4.7.2 System Requirements 56
4.7.3 Navigation & UI Overview 56
4.7.4. Core Workflows 57
Chapter Five Conclusion and Recommendation 59
5.1 Conclusion 59
5.2 Recommendation 59
5.3 Summary 60
5.4 References 61
5.5 Appendix 61

List of Tables

Table 1: Development Tool 15
Table 2: Team composition 16
Table 3: Use case table 22
Table 4: User table 42
Table 5: Student table 43
Table 6: Staff table 43
Table 7: Role Table 43
Table 8: User Role Table 43
Table 9: Seat Table 43
Table 10: Book Table 44
Table 11: Reservation with Book 44
Table 12: Reservation Without Book 44
Table 13: Wishlist Table 45
Table 14: Notification Table 45
Table 15: Test result 52

List of Figures

Figure 1: Use case diagram 21
Figure 2: Sequence diagram for login 23
Figure 3: Sequence diagram for student2 (student reserved seat without Book) 24
Figure 4: Sequence diagram for student reserved seat with Book 25
Figure 5: State Chart for Login 26
Figure 6: Activity diagram for login 27
Figure 7: Activity diagram for student (reserved Seat Book) 28
Figure 8: Activity diagram for student (reserved Seat without Book) 29
Figure 9: Class diagram 30
Figure 10: deployment diagram 33
Figure 11: The architecture design 34
Figure 12: Subsystem decomposition diagram 36
Figure 13: Component diagram 38
Figure 14: persistent data management 39
Figure 15: ER diagram 45
Figure 16: Library staff dashboard 1 49
Figure 17: Admin panel (Dashboard) 49
Figure 18: Library staff dashboard 2 50
Figure 19: Library staff dashboard 3 50
Figure 20: Backend test (Mocha test) 52
Figure 21: Jest test 53

Chapter One: Introduction

Libraries play a vital role in student life, offering essential spaces for learning, collaboration, and academic growth. Despite their importance, traditional library systems often fail to address the specific needs of students. Many libraries rely on manual processes for seat management, resource tracking, and notifications, which can hinder students' ability to use these spaces effectively. For students like myself, these inefficiencies create barriers to managing study schedules, accessing resources, and finding available seats, especially during peak times.

The StudyNest Library Application aims to address these challenges by providing a seamless, digital-first solution that redefines the student-library interaction. Designed with students' needs in mind, the app offers features like smart seat booking, real-time notifications, and personalized study productivity tools. Through these features, the app enables students to maximize their time and resources, making it easier to check seat availability, reserve spaces in advance, and receive updates on resources and library events—all through an intuitive mobile platform.

This project is inspired by my personal experiences as a student facing difficulties in managing time and library resources effectively. By tackling these common challenges, StudyNest seeks to empower students with tools that simplify their library experience and enhance their academic success. More than just a tool for convenience, the app strengthens the connection between students and libraries, ensuring these essential spaces remain organized, accessible, and supportive.

The StudyNest Library Application represents a significant step forward in bridging technology with education. By prioritizing the needs of students and integrating innovative digital solutions, the app transforms libraries into modern academic hubs, contributing to better time management, more efficient resource utilization, and overall student well-being

1.1 *Background Information of the Organization*

The organization behind the StudyNest Library Application is dedicated to transforming the way libraries operate by integrating technology into their systems. Libraries have always been central to academic success, providing students and faculty with critical resources and study spaces. However, traditional library systems, which still rely on manual processes, often fail to meet the needs of today's users. With increasing numbers of students and advancements in technology, there is a growing demand for a more efficient, technology-driven approach to library management.

The organization recognizes these challenges and aims to address them through the creation of the StudyNest Library Application, a solution designed to streamline library operations and improve user experience. By introducing automation, real-time updates, and user-friendly interfaces, the organization seeks to enhance the accessibility, efficiency, and functionality of library services.

1.1.1 Vision of the Organization

The vision of the organization is to revolutionize library services by creating a digital ecosystem that provides seamless access to library resources, real-time seat availability, and integrated study tools. By leveraging technology, the organization aspires to foster a productive, user-centered environment that supports students' academic success. Through this vision, the organization aims to position libraries as modern hubs of learning, collaboration, and resource optimization.

1.1.2 Mission of the Organization

The mission of the organization is to empower students and library staff by providing an innovative and automated solution to library management. The StudyNest Library Application aims to improve seat utilization, minimize time wastage, and enhance resource tracking through real-time data and intelligent systems. By automating manual processes and offering tailored services based on user needs, the organization is committed to creating an environment where students can maximize their academic potential, improve their study habits, and optimize their use of library resources. Additionally, the organization strives to support library staff by streamlining administrative tasks and providing real-time insights into library operations, ultimately fostering a more efficient and organized library experience.

1.2 *Background of the Project*

The StudyNest Library Application addresses the inefficiencies of traditional library systems, offering a digital solution for seat management, book rentals, study tracking, and peer collaboration. Many libraries still rely on outdated manual processes, causing frustration and wasted time, especially during peak periods like exams. Inspired by personal challenges in managing study schedules and resources, this app streamlines seat booking, provides real-time updates on seat and book availability, and tracks study sessions for improved productivity.

Beyond seat management, the app integrates a digital book rental system, study tools to monitor academic progress, and a community chat for peer support and collaboration. This solution empowers students to efficiently manage their time and resources, ultimately enhancing their academic success and fostering a more collaborative, modern library experience.

1.3 *Motivation*

The StudyNest Library Application was developed to address inefficiencies in traditional library systems and offer more than just a traditional solution. It is also developed to create safe space for Students in university. It provides a real-time seat reservation system, an integrated digital book rental service, study tracking tools, and a chat community for collaborative learning. These features aim to improve seat availability, streamline resource access, enhance productivity, and foster peer support, creating a more efficient and engaging academic environment. The application goes beyond traditional library management to empower students and improve their overall academic experience.

1.4 *Statement of the problem*

Students often encounter significant challenges in effectively utilizing libraries to support their academic success. Traditional library operations, relying on manual systems and outdated practices, do not meet the evolving needs of modern students. These limitations impact students' productivity, access to resources, and overall academic experience.

1. **Seat Availability Challenges**
    - Students frequently face difficulties identifying available seating, especially during peak usage times, resulting in wasted time and frustration.

- The lack of detailed seat information, such as location, availability, and amenities, further complicates the seating process.

2. **Lack of Timely Updates**
    - Libraries struggle to provide real-time updates on critical information, such as operating hours, resource availability, and library closures.
    - The absence of centralized notifications leads to missed opportunities, which could help students plan and optimize their library visits.
3. **Ineffective Study Support**
    - Students lack tools to track their study habits, manage time effectively, and set academic goals.
    - There are no personalized insights or reminders to help students maintain consistent study practices, resulting in reduced productivity.
4. **Manual Resource Management**
    - Libraries still rely on manual systems for managing books and other resources, leading to inefficiencies, delays, and errors.
    - Students are unable to track borrowed items, create digital wish lists, or receive automated notifications about the availability of resources.
5. **Inefficient Space Utilization**
    - Library seats often remain unnecessarily occupied during long breaks or idle periods, reducing the overall efficiency of space usage.

1.5 *Objectives of the Project*

1.5.1 General Objective

The To develop an efficient, automated seat management system that enhances library resource utilization by integrating study tracking, user classification, book rental monitoring, and real-time data analytics.

1.5.2 Specific Objectives

- **Design a user-friendly mobile and web-based interface** to facilitate manual seat ID input and cancellation, ensuring ease of access and smooth interaction with the system.
- **Implement a remote seat availability checker** to help students plan library visits more

effectively, minimizing unnecessary travel and congestion.

- **Classify users by usage type (e.g., regular, short-term, research-focused)** to tailor services and allocate library resources more efficiently.
- **Develop an admin dashboard to provide real-time insights** into seat occupancy, user activity, and book rentals, enabling informed operational decisions.
- **Incorporate a study session tracker** that allows students to log and review their study hours, fostering better time management and academic discipline.
- **Status features** Notifications about unexpected closures, status of the library (e.g. closed for holiday) ensuring users are promptly informed.
- **Announcements** for scheduled changes, upcoming events, and general library updates.
- **Integrate customizable countdown timers** for managing personal study and break intervals, encouraging healthier and more productive study habits.
- **Utilize tracking data for optimizing resource allocation** such as seating, staffing, and book handling, ensuring efficient use of physical and digital assets.
- **Provide advanced analytics tools** to generate actionable insights for continuous system improvement and strategic planning by administrators.
- **Support Service** enables users to report issues such as damaged chairs, desks, or malfunctioning equipment directly through the app
- **Community Hub** serves as a department-based communication space

1.6 *Feasibility Analysis*

1.6.1 Operational Feasibility

The proposed application is designed to address existing challenges faced by students and library users by offering accessible, student-centric features that enhance productivity and streamline library operations. The app will provide functionalities such as book search, reservation, due date reminders, and access to digital resources, all tailored to meet the needs of its users. By simplifying library processes and improving accessibility, the app is expected to integrate seamlessly into the daily routines of students and library staff, ensuring high operational feasibility.

1.6.2. Technical Feasibility

The system will be developed using React Native for the frontend, ensuring a consistent and

responsive user experience across multiple platforms (iOS, Android, and web). The backend will be powered by **Mongo DB**, which provides real-time database capabilities, authentication, and cloud storage, making it ideal for handling dynamic data and user interactions. GitHub will be used for version control, enabling efficient collaboration among developers and ensuring code integrity. These technologies are well-documented, widely supported, and suitable for building scalable, cross- platform applications, confirming the technical feasibility of the project.

1.6.3. Economic Feasibility

The development costs for the project are minimal, as the primary expenses are limited to software tools, hosting services, and occasional maintenance. React Native and Mongo DB offer free tiers that are sufficient for initial development and testing, while GitHub provides free repositories for version control. Hosting costs for Mongo DB are scalable and cost-effective, ensuring that the project remains economically viable. Additionally, the long-term benefits of improved library operations and enhanced student productivity outweigh the initial investment, making the project financially feasible.

1.6.4. Behavioral Feasibility

The app is expected to gain widespread acceptance among its target users, primarily students and library staff, due to its focus on resolving common challenges such as book availability, due date tracking, and access to resources. The user-friendly interface and intuitive design will ensure ease of use, while features like personalized reminders and notifications will enhance user engagement. By addressing real pain points and providing practical solutions, the app is likely to be well-received and adopted by its intended audience.

1.6.5. Schedule Feasibility

The project will be executed in well-defined phases to ensure timely development and testing. Phase 1 will focus on requirement analysis, system design, and the development of core functionalities. Subsequent phases will involve iterative testing, user feedback collection, and the implementation of additional features. A detailed project timeline with milestones and deadlines will be established to monitor progress and ensure that the project stays on track. This phased approach ensures that the project is completed within the allocated time frame.

1.6.5. Economic Feasibility

Economic feasibility assesses whether the benefits of the project outweigh its costs, ensuring it is a sound financial investment.

1.6.5.1 Cost-Benefit Analysis

This project offers several tangible and intangible benefits that justify its costs:

1. Benefits:
    - Operational Efficiency: Automates library operations like seat booking and book rentals, reducing manual workload.
    - Revenue Opportunities:
        - Subscription fees from users.
        - Partnerships with publishers or institutions for promotion and integration.
        - In-app advertisements.
    - Scalability: Designed to accommodate multiple institutions, ensuring broader adoption and usage.
    - User Satisfaction: Enhanced user experience through intuitive design and real-time seat availability.
2. Costs:
    - Initial Development Costs: Hardware, software tools, and app store publishing fees.
    - Recurring Costs: Cloud hosting, database management, and maintenance.

For the app to be economically viable, its cumulative revenue (from subscriptions and other sources) and cost savings must exceed the total cost of development and operations.

1.6.5.2 Cost of the Project

1. Hardware

- **Mobile Devices and Testing**: 0–60,000 Birr
  For testing across iOS/Android devices, using either existing or new equipment.
- **Server Hardware**: 25,000–62,500 Birr
  For hosting backend, database, and storage via servers or cloud platforms.

2. Software

- **Development Tools**: 0 –12,500 Birr
  Tools like vscode and Android Studio for app creation.
- **Backend Tools**: 0–25,000 Birr

Frameworks like Node.js for server-side logic and database integration.

- **Database Software**: 0–12,500 Birr/month
  For storing user data (MySQL, MongoDB, Mongo DB, etc.).
- **Cloud Hosting**: 2,500–12,500 Birr/month
  For app hosting and scalable storage.
- **Design Software**: 0–6,250 Birr/month
  UI/UX design tools (e.g., Figma, Adobe XD).

3. Resources

- **Learning Materials**: 6,250–25,000 Birr
  Books or online courses for app development.
- **API Documentation**: 0–12,500 Birr
  For APIs enabling notifications, seat booking, etc.

4. Testing

- **Testing Tools**: 6,250–12,500 Birr/month
  Services like Mongo DB Test Lab for cross-platform compatibility.

5. Miscellaneous

- **Domain & Hosting**: 2,500–12,500 Birr/year
  For a website or landing page hosting.
- **App Store Fees**: 24,875 Birr
  Publishing fees for iOS and Android.

1.6.6 Cost Breakdown

One-Time Costs

- **Hardware**: 0–60,000 Birr (testing devices) and 25,000–62,500 Birr (server setup).
- **Development Tools**: 0–12,500 Birr (software like Xcode, Android Studio).
- **Learning Resources**: 6,250–25,000 Birr (books, online courses).
- **App Store Fees**: 24,875 Birr (initial publishing fees).

  Total One-Time Costs: **56,125 Birr –** 184,875 **Birr**

Total Recurring Costs:

Monthly Costs:

- **Cloud Hosting & Storage:** 2,500–12,500 Birr

- **Database Management:** 0–12,500 Birr
- **Testing Tools:** 6,250–12,500 Birr
- **Total Monthly Cost:** 8,750–37,500 Birr/month

Annual Costs:

- **Miscellaneous (Domain/Hosting):** 2,500–12,500 Birr/year

1.7 *Scope of the Project*

The StudyNest Library Application is designed as a mobile and web-based platform to improve student interactions with library resources. The application focuses on providing essential features such as seat management, enabling students to check and reserve library seats in real-time. It also includes a streamlined book rental system for efficient borrowing and returning of books. Additionally, the app offers study tracking tools to monitor study sessions and academic progress, along with real-time announcements to keep students informed about library events and resource availability. This student-centered solution aims to simplify library usage, enhance productivity, and support academic success

1.8 *Significance of the project*

The StudyNest Library Application addresses critical challenges faced by students in accessing and utilizing library resources, significantly improving their academic experience. This study proposes a digital solution to modernize library systems, making them more efficient, accessible, and student-centered.

Key Contributions:

1. **Improving Academic Productivity**
   The application enables students to manage their study time effectively through real-time seat reservations, personalized study tracking, and motivational feedback, leading to enhanced academic performance.

2. **Enhancing Student-Resource Interaction**
   Features like book rental systems and real-time availability updates simplify the process of finding and renting books, eliminating manual tracking and improving resource accessibility.

3. **Fostering Collaborative Learning**
   The department-based community hub facilitates peer interaction, allowing students to share study tips, seek academic help, and build a collaborative support network, particularly for challenging subjects.

4. **Optimizing Library Space Utilization**
   Real-time seat tracking and break time management improve library space utilization, ensuring fair allocation and reducing unnecessary seat occupancy.

5. **Contributing to Digital Transformation in Education**
   By showcasing how technology enhances library services, this study provides a model for other educational institutions to adopt digital solutions that improve student experiences and academic success.

6. **Promoting Inclusivity and Accessibility**
   Accessibility features such as dark mode, screen reader compatibility, and multi-language support ensure that students, including those with disabilities and diverse linguistic backgrounds, can benefit from the app, fostering a more inclusive academic environment.

7. **Supporting a Multilingual Academic Community**
   By offering a **multi-language interface**, the app caters to students from diverse linguistic backgrounds, ensuring they can access library resources in their preferred language. This feature enhances usability and engagement, particularly in multilingual academic institutions.

1.9 *Target beneficiaries of the system*

The StudyNest Library Application is designed to benefit a wide range of users and stakeholders in the academic ecosystem

Students

- **Undergraduate and Graduate Students**: Gain access to real-time seat reservations, book availability, and a collaborative community hub.

Library Staff

- Streamlines library management by automating book rentals, seat tracking, and usage

analytics

Educational Institutions

- Modernizes library systems and improves student satisfaction

1.10. *Methodology*

Our methodology for developing the StudyNest Library Application adopts a modular approach, breaking the project into smaller, manageable components or "modules." This approach ensures greater flexibility and allows for easier adaptation during the development process. The methodology consists of the following key steps:

1. Module Identification

- The first step is to identify and define the core modules within the application. Each module serves a specific function that collectively contributes to the overall system. In the context of the StudyNest Library Application, these modules include:
- Seat Availability Management: Handles seat reservations, availability checks, and preferences.
- Announcements System: Manages notifications and updates about the library's operations.
- Break Time Management: Tracks study sessions and enforces break durations for fair seat use.
- Study Tracking System: Tracks student productivity, including session logs and performance analytics.
- Book Rental System: Manages book availability, reservations, and wish lists for library materials.

2. Requirement Gathering at the Module Level

Each module is analyzed independently to gather its specific requirements. This involves engaging with stakeholders, including students, library staff, and IT professionals, to understand the unique needs and challenges of each module. For example, the Seat Availability Management module requires input on how students reserve seats, check seat status, and manage seat preferences.

3. Design and Prototyping at the Module Level

Once requirements are gathered, design and prototyping are carried out for each module. This includes:

- Defining the architecture of each module.
- Designing the user interfaces (UI) and user experience (UX) for easy navigation and interaction.
- Establishing the database structure and API specifications.
- Creating mockups and prototypes to visualize how each module will function.

4. Development and Testing at the Module Level

Development and testing are performed on a module-by-module basis. This allows for more focused efforts on specific functionalities, such as implementing the real-time seat reservation system or the performance tracking analytics. Each module undergoes rigorous testing to ensure it operates correctly, focusing on:

- Unit testing to validate functionality.
- User acceptance testing (UAT) to verify that the module meets the user requirements.
- Performance testing to ensure scalability and responsiveness.

5. Integration and Interoperability

After individual modules are developed and tested, they are integrated into a cohesive system. The integration phase ensures that the modules work together seamlessly. Integration testing is performed to ensure smooth data flow between modules, such as ensuring that the Book Rental System integrates properly with the Seat Availability Management system, allowing students to check out books and reserve seats without issues.

6. Deployment and Maintenance

Once the system is fully integrated and tested, it is deployed to a reliable hosting platform. Deployment involves setting up the app on the cloud, ensuring that it is accessible to users, and ensuring all backend processes are functional. A maintenance plan is established to provide ongoing support, fix bugs, and update the system as needed. Maintenance occurs at both the individual module level (e.g., updating the Announcements System) and the system-wide level (e.g., performance improvements and new features).

7. Project Evolution

The performance and impact of the StudyNest Library Application are continuously monitored. Feedback from students, faculty, and library staff is collected to identify areas of improvement. This includes gathering data on how the Break Time Management system is impacting study habits.

- The effectiveness of the Study Tracking System in motivating students.
- User engagement with the Book Rental System and seat reservation features.

This feedback is used to enhance and evolve the application, ensuring it remains responsive to user needs. New features or improvements are planned and added in future versions of the app, based on real-world usage and stakeholder input

1.10.1 Fact-Finding Techniques

Fact-finding is a critical aspect of system development, as it helps gather the necessary information for system design and development. The following techniques will be used to gather information for the StudyNest Library Application:

Interview

Interviews with key stakeholders, such as students, library staff, and faculty members, will be conducted to gather qualitative data on user needs and challenges. These interviews will help understand expectations for the system, including specific requirements for features like seat management, book rentals, and study tracking.

Practical Observation

Observation of current library operations will allow the development team to identify pain points in existing workflows and interactions. This can provide valuable insights into how users interact with the library's resources, and what improvements can be made through the new system.

Document Analysis

Existing library documentation, such as policies, operational guidelines, and usage reports, will be analyzed to understand current processes. This helps ensure the new system aligns with library standards and regulations and can support future developments effectively.

1.10.2 Systems Analysis and Design

The StudyNest Library Application will go through the following steps:

- Systems Analysis: Involves understanding the problem, identifying the needs of users, and outlining the functional requirements of the system. This will include mapping out workflows, user interactions, and system functionalities for each module, such as the seat management, announcements, study tracking, and book rental systems.
- Systems Design: Based on the analysis, the design phase will focus on creating the architecture, databases, user interfaces (UI), and defining the system's backend structure.

This ensures all components of the system work together seamlessly, with user-friendly features to meet the needs of students and library staff.

1.10.3. Development Tools

category	Activities	Tools/programs/Hard war e	Description
Software Tools	Platform	windows	Operating system for app development and testing.
	Mobile Development	Android Studio	Tools for building the mobile app for Android and iOS platforms.
	Backend Development	Node.js,	Backend server and real-time database for managing seat availability, announcements, and study data.
	User Interface Design	Figma	For designing app UI prototypes
	Browsers	Google Chrome, Mozilla Firefox	Used for testing web-based features or debugging APIs.
	Documentation	Microsoft Word, Microsoft Excel	For documenting project reports, database structures, and project planning.
	Code Editors	Visual Studio Code	Lightweight editors for writing and debugging the code.
	Version Control	GitHub	For version control and collaborative coding.
	Testing Tools	Postman	Used to test and debug RESTful APIs for backend integration.

Table 1: Development Tool

1.11. *Limitation of the project*

While StudyNest offers some offline functionality, full features such as real-time seat availability tracking, book rental updates, and timely announcements require an active internet connection. The application is designed for mobile devices, which may limit access for students using older devices or unsupported operating systems. Additionally, the app is focused on providing solutions for a specific set of library services, and features beyond seat reservations, book rentals, and study tracking may not be included in the initial release.

1.12 Team composition

Prepared By:	የጥናት ጎጆ | **StudyNest**			
	S No.	Full Name	Id No	Email/Mobile
	1.	Hailemichel Eshetu	RU1090/14	eshetuhailemichael06@gmail.com
	2.	Yusra Reshid	RU0048/14	Yusramuhaba8@gmail.com
	3.	Changkuoth Top	RU0348/14	gchangkuothtop@gmail.com
	4.	Abdurahman Seid	RU0234/14	abduseid8@yahoo.com
	5.	Tsion Argaw	RU1610/14	Tsionargaw143@gmail.com
Date:	May 6, 2025			

Table 2: Team composition

Chapter Two: Requirement Analysis and Specification

2.1 *Current System*

The current system for managing student-library interactions at educational institutions is outdated, primarily reliant on manual processes. Students face significant challenges, such as difficulties in finding available seats, delays in accessing books, and ineffective communication about library events or closures. The lack of a centralized digital platform limits transparency and accessibility, making it harder for students to manage their academic needs efficiently.

Furthermore, current systems fail to provide real-time updates, study tracking tools, or collaborative features that modern students require to optimize their academic performance. Libraries often struggle with space utilization, leading to inefficiencies and dissatisfaction among users. While existing systems might meet basic operational needs, they fall short of creating an engaging and productive environment for students.

The StudyNest Library Application aims to address these gaps by providing a centralized, user-friendly digital platform tailored to student needs. While traditional face-to-face interactions remain integral to library operations, the proposed system will enhance these processes with features like real-time notifications, seat reservations, and personalized study tools. This transformation will ensure transparency, accessibility, and convenience, fostering a better academic environment.

2.2 *Business Rules*

The StudyNest Library Application will adhere to the following business rules to ensure efficient and fair operations:

- **User Verification:** All users must undergo a secure registration and verification process to access the platform.
- **Seat Management:** Users can reserve and release library seats through the app, ensuring efficient utilization.
- **Role-Based Access:** Different user roles, such as students, library staff, and administrators, will have tailored functionalities to suit their needs.

- **Resource Tracking:** The system will provide real-time updates on book availability, due dates, and rental statuses.
- **Study Session Monitoring:** Users can log and track their study sessions, receiving insights to improve productivity.
- **Announcement Management:** Administrators can post updates about library events, closures, or new resources.
- **Break Time Allocation:** A countdown system ensures that seats are not unnecessarily occupied during breaks, optimizing space usage.

These business rules aim to create a secure, efficient, and user-centric environment that enhances student-library interaction and supports academic success.

2.3 *Proposed System*

2.3.1 Overview

The StudyNest Library Application is designed to revolutionize how students interact with library resources. By providing a digital-first approach, the system will streamline seat reservations, resource access, and study tracking. The app will also include a community hub to foster peer support and collaboration.

The platform will cater to students, library staff, and administrators, integrating their roles into a cohesive system that ensures transparency, efficiency, and user satisfaction. By addressing the shortcomings of current systems, StudyNest will enhance library operations and student productivity.

2.4 *Requirements of the Proposed System*

2.4.1 Functional Requirements

The system is designed to meet the following functional requirements:

1. **User Registration and Authentication**
    - Feature: Users can register and log in with secure credentials.
    - Priority: High
2. **Seat Reservation and Management**
    - Feature: Students can reserve, release, and check seat availability.

    - Priority: High
3. **Book Rental Management**
    - Feature: Users can search for, borrow, and return books through the app.
    - Priority: High
4. **Study Session Tracking**
    - Feature: Users can log study sessions and receive productivity analytics.
    - Priority: Medium
5. **Announcements**
    - Feature: Administrators can post updates visible to all users.
    - Priority: Medium
6. **Community Hub**
    - Feature: A department-based chat system for peer collaboration.
    - Priority: Medium

User Requirements

High Priority:

- UR-01: Secure user registration and authentication.
- UR-02: Real-time seat reservation and release functionality.
- UR-03: Comprehensive book rental management system.
- UR-04: Administrator access for managing announcements and resources.

Medium Priority:

- UR-05: Study session tracking with analytics.
- UR-06: Department-based community chat for collaboration.

2.4.2 Non-Functional Requirements

1. **Performance:** The system will ensure quick data retrieval, minimizing delays.
2. **Accessibility:** Available on multiple platforms, including mobile and web.
3. **User Interface:** A clean, intuitive design for seamless navigation.
4. **Reliability:** High uptime and robust error handling to ensure consistent availability.

5. **Security:** Comprehensive measures, including encryption and user authentication, to protect data.
6. **Scalability:** The system will accommodate growing user bases and additional features.

2.5 *System Analysis*

An analysis model is a conceptual representation used in the early stages of software development to understand, describe, and document the requirements and behavior of a system. It serves as a visual or mathematical representation of different aspects of the system, allowing stakeholders, including analysts, designers, and end-users, to comprehend the system's structure, functionality, and interactions. Analysis models are instrumental in capturing and communicating complex information in a clear and organized manner. The analysis model for our system employs various diagrams to comprehensively depict the system's structure, behavior, and interactions. This model serves as a foundational tool for understanding and communicating the intricacies of the system's processes, ensuring alignment with the requirements.

2.5.1 Use case model diagrams

A Use Case Diagram visually represents the interactions between actors (users) and the system, highlighting the system's functionalities from an external perspective. It provides a high-level overview of how the student, library staff, and admin interact with the system and the various use cases that define these interactions. Key use cases may include student registration, book borrowing, book return

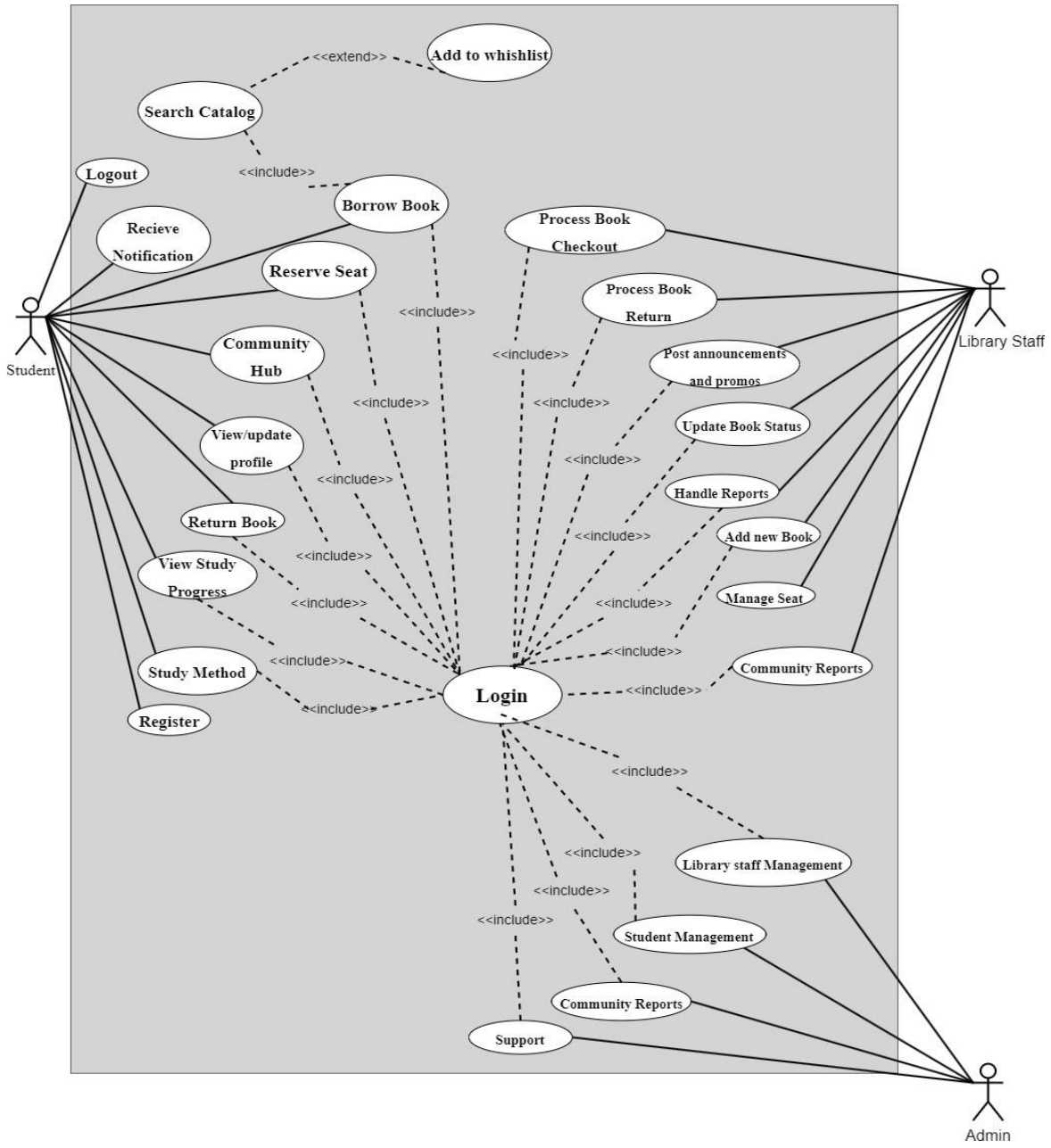


Figure 1: Use case diagram

UCID	UC Name	UC Description
UC-1	Register	Allows students to create an account in the system.
UC-2	Login	Allows students and library staff to log in to the system.
UC-3	Logout	Allows students and library staff to log out of the system.
UC-4	Search Catalog	Enables students to search for books in the library catalog.
UC-5	Add to Wishlist	Allows students to add books to their Wishlist (extends Search Catalog).

UC-6	Borrow Book	Allows students to borrow books from the library.
UC-7	Return Book	Allows students to return borrowed books.
UC-8	View/Update Profile	Enables students to view or modify their personal information.
UC-9	View Study Progress	Allows students to track their academic progress.
UC-10	Study Method	Provides students with study techniques or resources.
UC-11	Receive Notification	Sends notifications to students (e.g., due dates, announcements).
UC-12	Remove Seat	Allows students to release reserved study seats.
UC-13	Process Book Checkout	Enables library staff to manage book borrowing transactions.
UC-14	Process Book Return	Allows library staff to handle book returns and update inventory.
UC-15	Post Announcements and Promos	Enables library staff to share updates, promotions, or announcements.
UC-16	Update Book Status	Allows library staff to modify book availability or condition.
UC-17	Handle Reports	Enables library staff to generate or manage system reports.
UC-18	Add New Book	Allows library staff to add new books to the catalog.
UC-19	Manage Seat	Enables library staff to oversee study seat allocations.
UC-20	Community Reports	Allows library staff to generate reports related to community activities.
UC-21	Library Staff Management	Enables library staff to manage staff accounts and permissions.
UC-22	Student Management	Allows library staff to manage student accounts and access.
UC-23	Support	Provides library staff with tools to resolve user issues.
UC-24	Community Hub	Facilitates interactions and shared activities within the community (system component).

Table 3: Use case table

2.5.2 Sequence diagram

A Sequence Diagram is a dynamic modeling tool that shows the interactions between different components of a system over time. It emphasizes the chronological order of messages exchanged between objects or entities

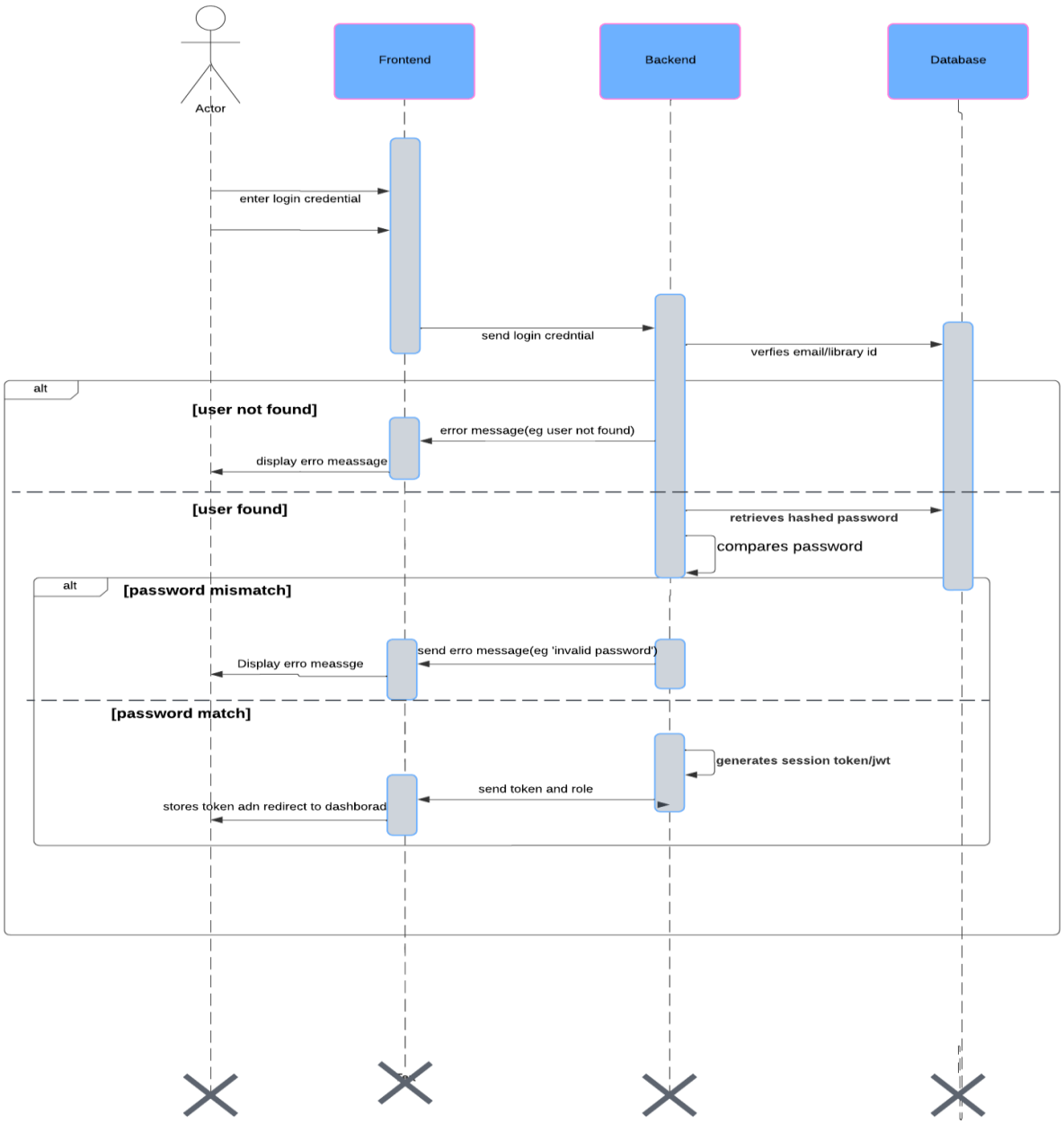


Figure 2: Sequence diagram for login

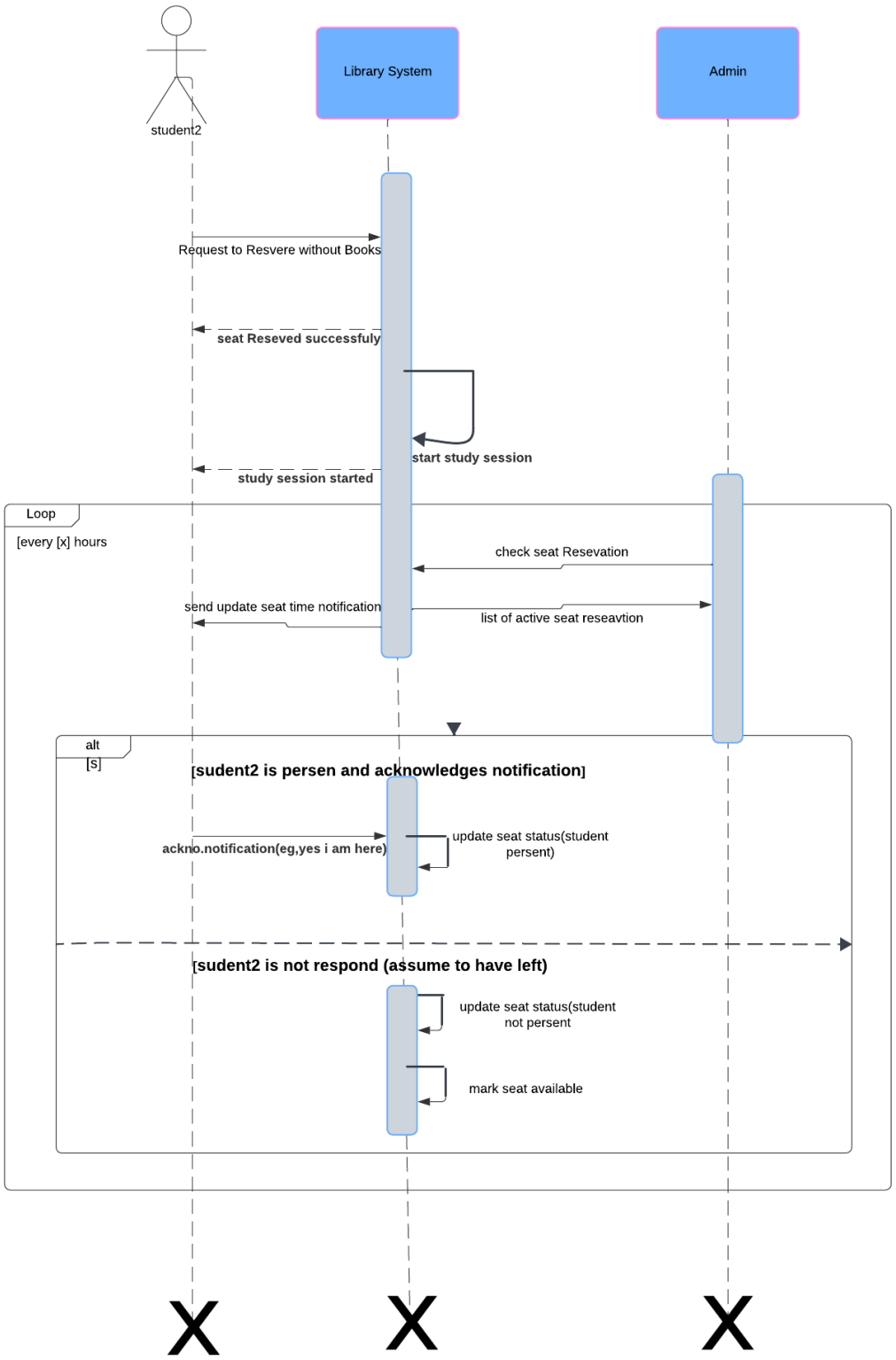


Figure 3: Sequence diagram for student2 (student reserved seat without Book)

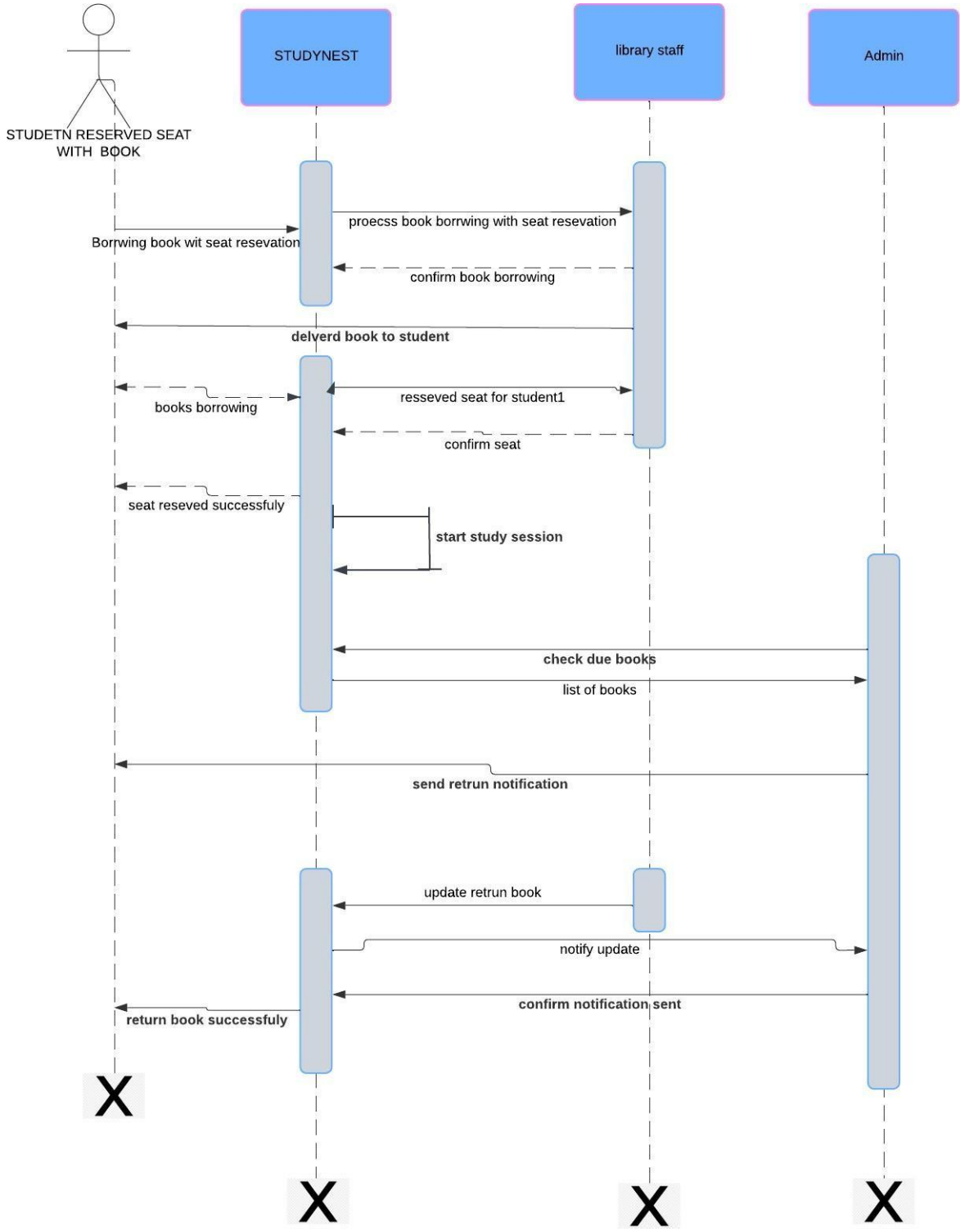


Figure 4: Sequence diagram for student reserved seat with Book

2.5.3 State chart diagram

A State Chart Diagram is a behavioral modeling tool that illustrates the various states an object or system can undergo and the transitions between these states. Within our system, the State Chart Diagram will represent the different states that entities, such as user accounts or requests, can exist in. It helps to visualize how these entities transition from one state to another in response to events or triggers, offering a clear understanding of the system's behavior under different conditions.

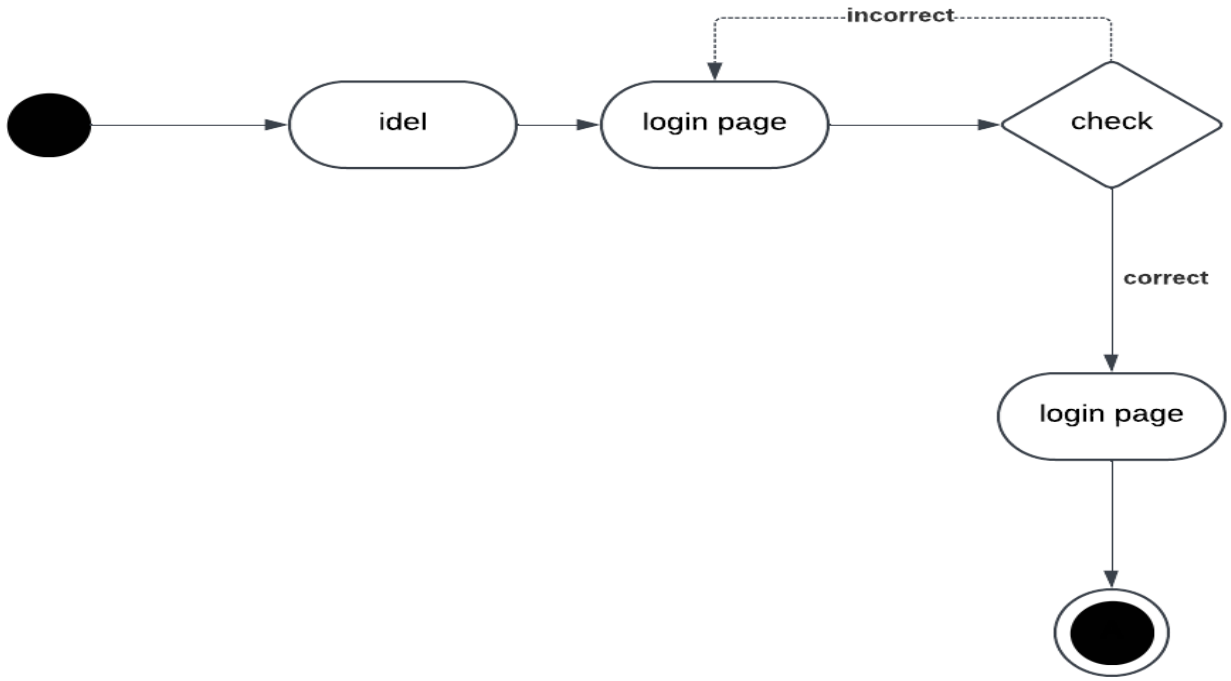


Figure 5: State Chart for Login

2.5.4 Activity diagram

The Activity Diagram is a behavioral modeling tool that visualizes the dynamic aspects of a system by illustrating the sequence of activities or actions within a process. It serves as an excellent means of representing the flow of tasks and actions, providing a step-by-step view of how different components interact over time. In the context of our project, the Activity Diagram helps to capture the dynamic behavior of processes such as user registration, login . the Activity Diagram enhances our understanding of the chronological order of events, aiding in the analysis and design

of the system's functionality.

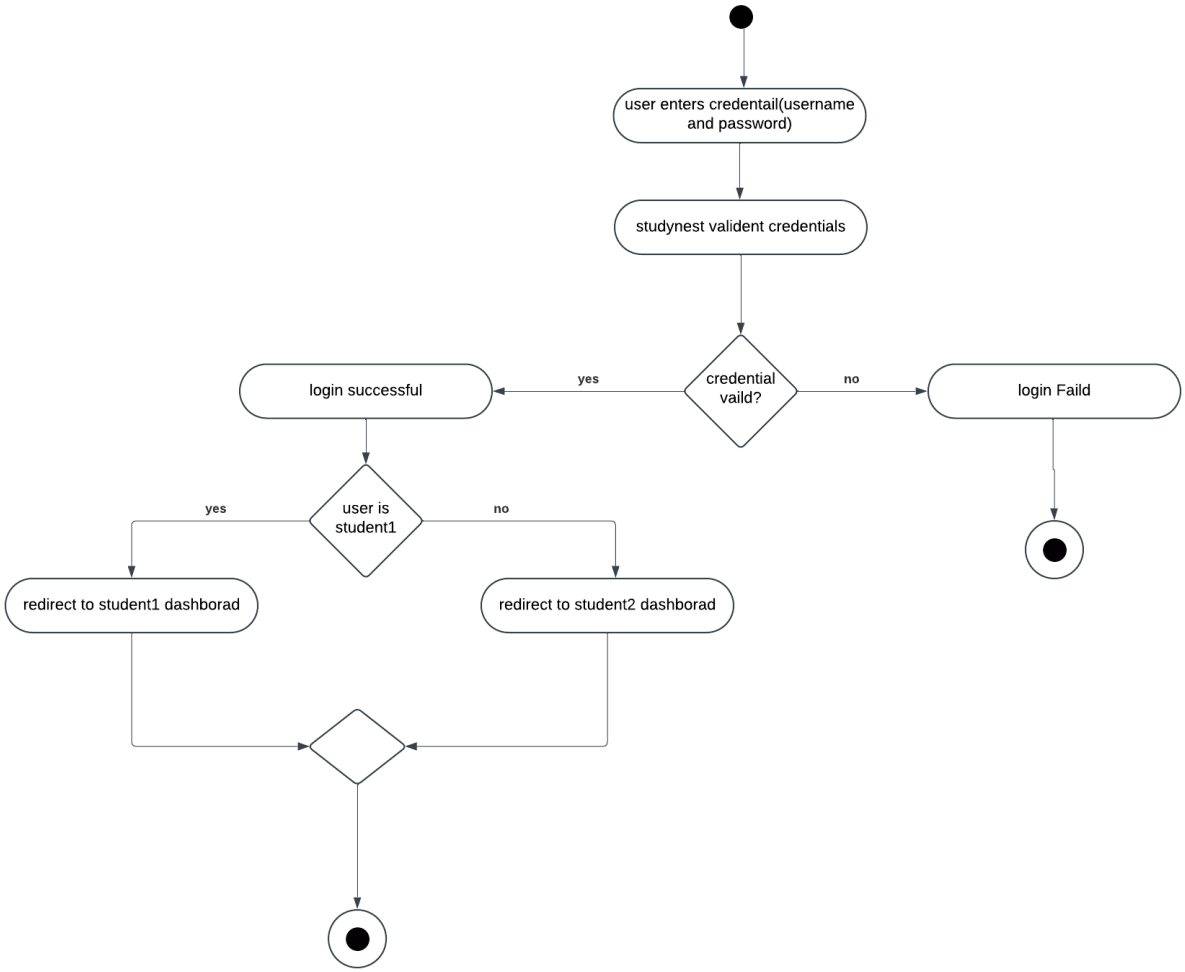


Figure 6: Activity diagram for login

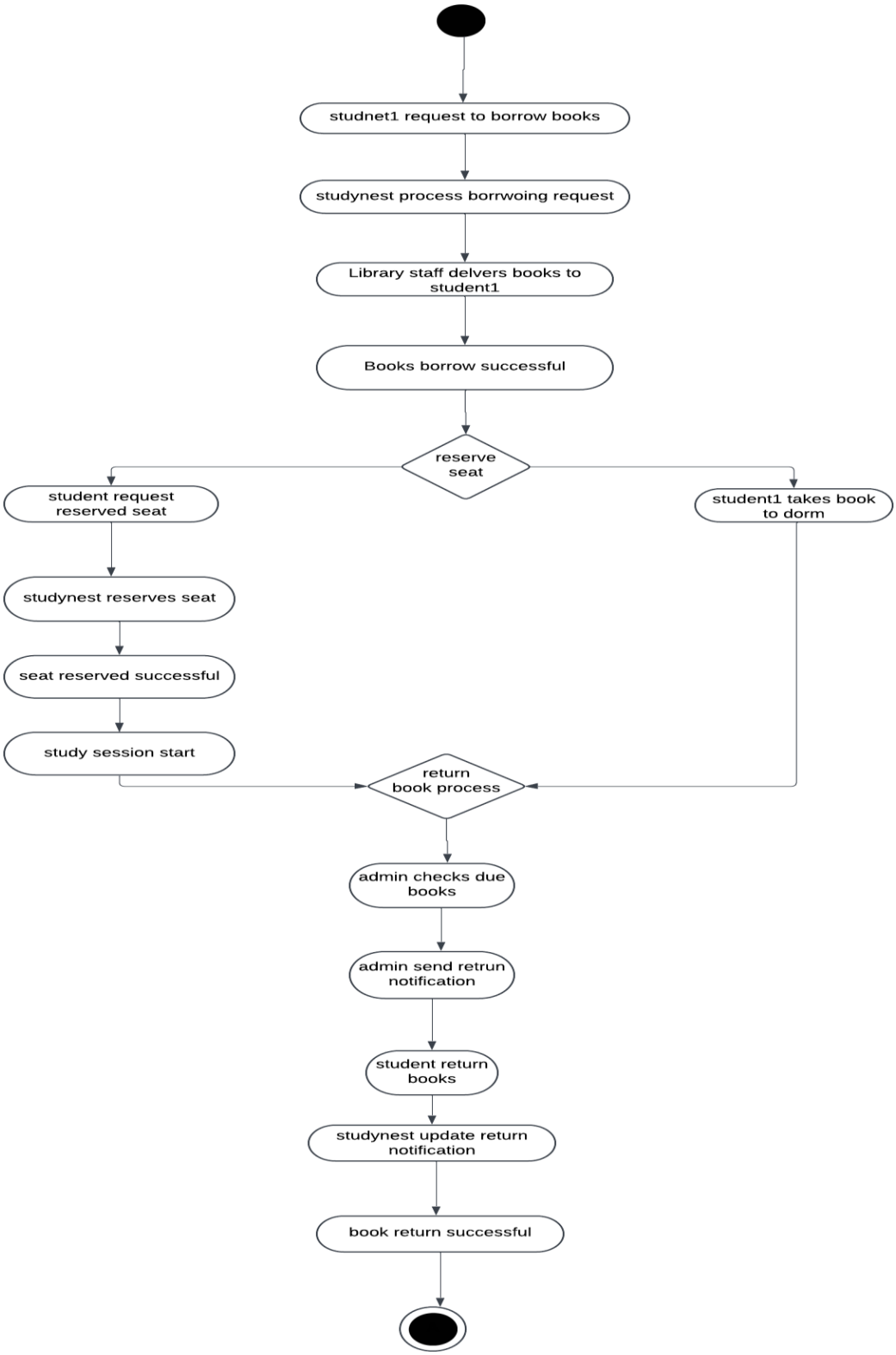


Figure 7: Activity diagram for student (reserved Seat Book)

Figure 8: Activity diagram for student (reserved Seat without Book)

2.5.5 Class diagram

A Class Diagram is a static modeling tool that depicts the structure of a system by illustrating the classes, attributes, and relationships between them

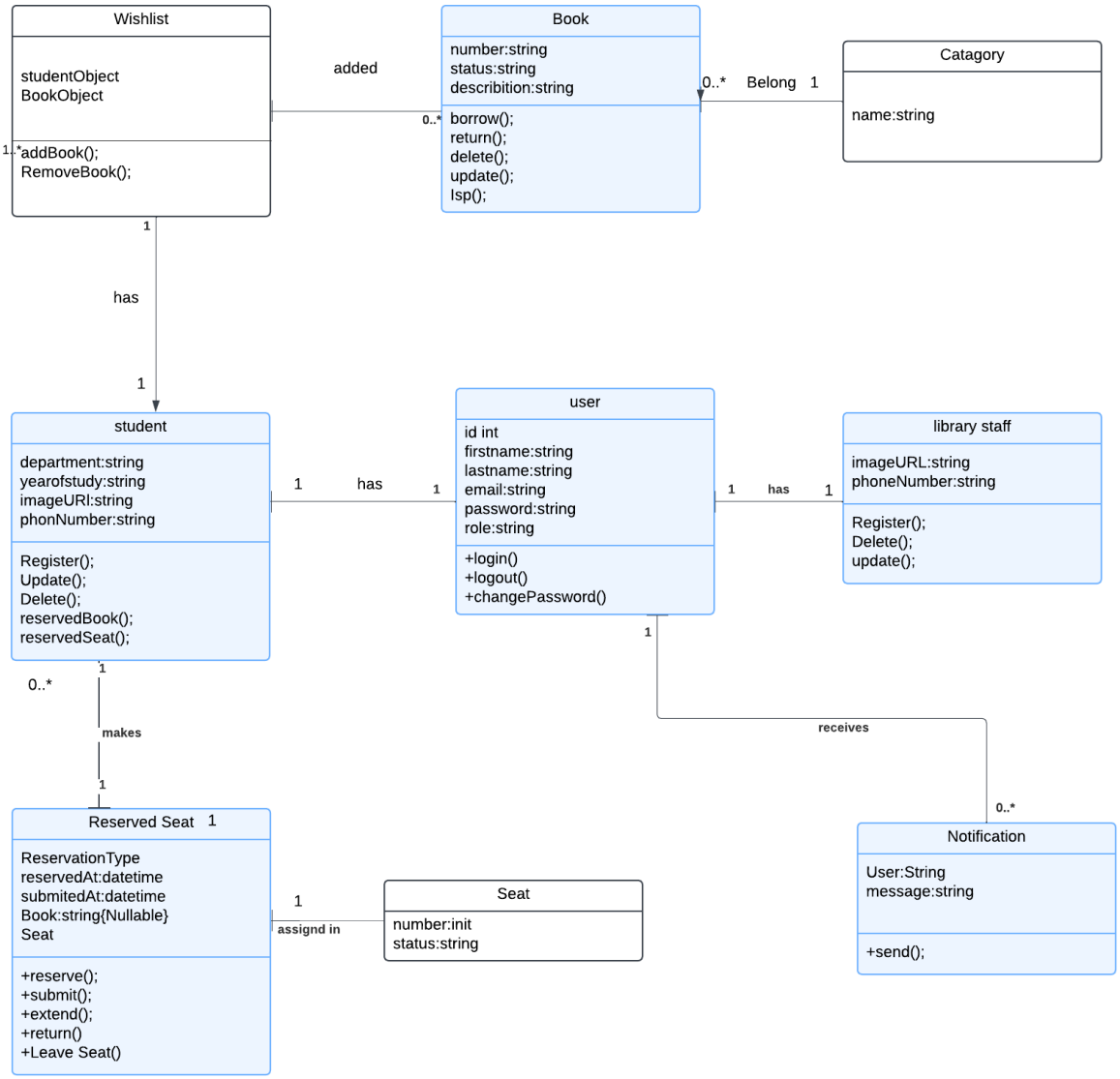


Figure 9: Class diagram

Chapter Three: System Design

3.1 *Introduction*

System design is a crucial phase in the software development life cycle where the requirements gathered during the analysis phase are transformed into a detailed design. It involves creating the architecture, components, interfaces, and data structures of the system, ensuring that it meets the functional and non-functional requirements identified earlier. The system design phase lays the foundation for the development and implementation of the system we propose.

3.2 *Purpose of the system*

The system design phase marks a critical step in the development of the proposed system, transitioning from conceptual understanding to a concrete blueprint. This phase is dedicated to crafting a well-structured and efficient system that not only meets the specified requirements but also anticipates future scalability and aligns with the overarching business goals.

Key Objectives:

- **Translating Requirements**: The primary objective is to convert the insights gained from the analysis model into a detailed and practical system design.
- **Defining Architecture**: Establishing the system's architecture, including components, modules, and their interactions, forms the foundation for a coherent and organized structure.
- **Ensuring Scalability**: Designing with scalability in mind ensures the platform can seamlessly grow to accommodate increased user demand and evolving functionalities.
- **Enhancing User Experience**: Prioritizing usability and crafting an intuitive interface contributes to an overall positive user experience, fostering engagement and satisfaction.
- **Addressing Security**: Implementing robust security measures is paramount to safeguarding user data, transactions, and the overall integrity of the system.
- **Facilitating Development**: The system design provides a clear roadmap and specifications, guiding the development team in the coding and implementation phases

3.3 *Design Goals*

The design of the StudyNest Library Application is guided by several overarching goals, each contributing to the creation of a robust and user-centric platform:

- User-Friendly Interface: Develop an intuitive and easy-to-navigate interface to accommodate students and library staff with varying levels of technical expertise. This ensures a positive and efficient user experience, enabling seamless access to library resources and features.
- Scalability: Design the system to scale effortlessly as the number of users increases, accommodating a growing student population, expanded library inventory, and the addition of new features or functionalities.
- Security and Privacy: Prioritize robust security measures to safeguard student data, including personal information, study logs, and book rental history, fostering a secure and trustworthy environment for all users.
- Efficient Data Flow: Establish efficient data flow mechanisms within the system to ensure real-time updates for seat availability, accurate book listings, and smooth communication between students and library staff.
- Library Integration: Seamlessly integrate library resources such as seat management, book inventory, and announcements into the application, fostering a collaborative and cohesive library ecosystem that enhances the overall experience for users.
- Personalization: Incorporate features that allow students to create personalized profiles, track their study sessions, manage their book rentals, and receive tailored notifications or recommendations, enhancing engagement and satisfaction.

3.4 *Deployment Diagram*

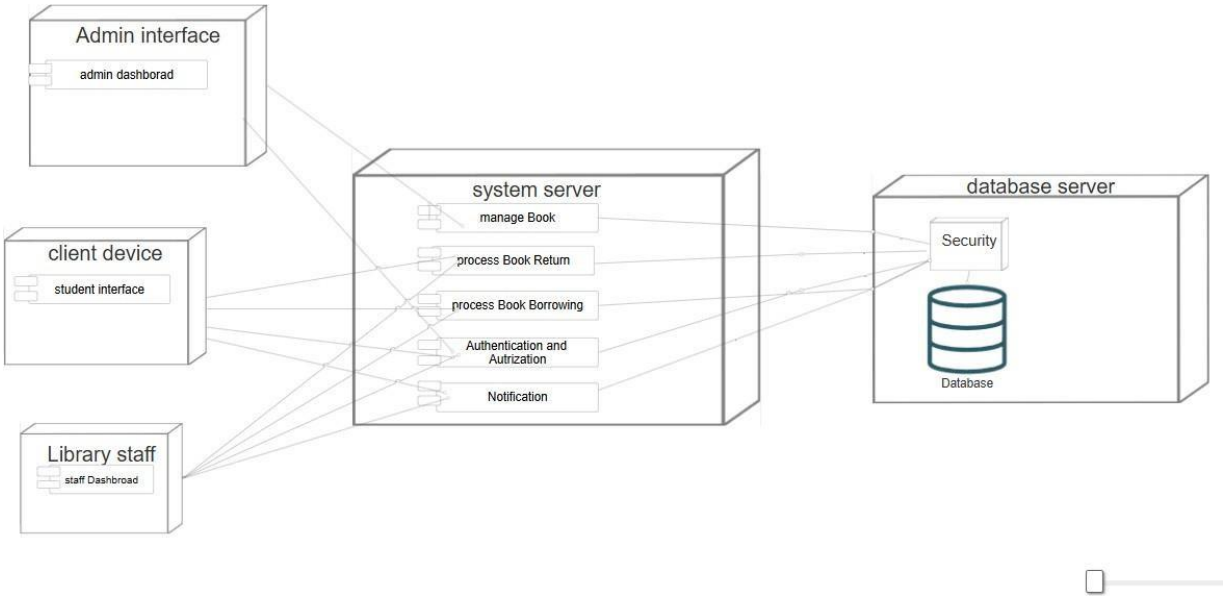


Figure 10: deployment diagram

3.5 *Architecture design*

Architectural Design for the proposed system involves structuring and organizing the system's components, defining their relationships, and specifying their functionality. This top-level design provides an overview of the entities that collaborate to achieve the system's goals. The architectural design for our system involves structuring and organizing its components to achieve efficiency, scalability, and functionality.

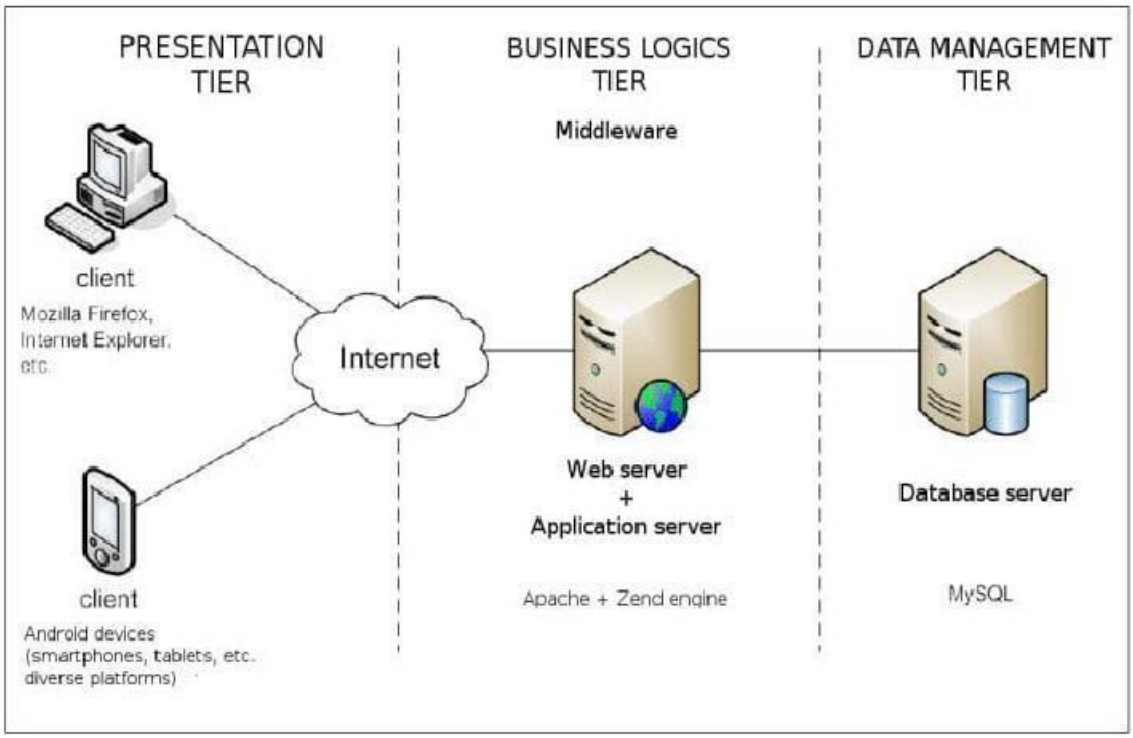


Figure 11: The architecture design

3.5.1. Subsystem decomposition

1. User Management Subsystem

- Responsibility:
  - Handles user registration, authentication, and profile management.
  - Manages roles (e.g., Student, Library Staff, Admin) and permissions.
- Key Features:
  - User registration and login.
  - Role-based access control (e.g., Student, Library Staff, Admin).
  - Profile management (e.g., updating personal information, password reset).

2. Book Management Subsystem

- Responsibility:
  - Manages the process of adding, updating, and removing books from the library catalog.
  - Handles book details (e.g., title, author, category, availability).

- Key Features:
    - Add, update, and remove books.
    - Search and filter books by title, author, or category.
    - Track book availability and status (e.g., available, reserved, borrowed).

3. Seat Management Subsystem

- Responsibility:
    - Manages the allocation and reservation of library seats.
    - Ensures seats are assigned based on availability and user preferences.
- Key Features:
    - Reserve seats for students.
    - Track seat availability and status (e.g., available, occupied).
    - Handle seat reservations with or without book reservations.

4. Reservation Management Subsystem

- Responsibility:
    - Manages book and seat reservations.
    - Handles the process of reserving books and seats, including deadlines and notifications.
- Key Features:
    - Reserve books and seats.
    - Track reservation deadlines and submission dates.
    - Send notifications for overdue reservations or upcoming deadlines.

5. Notification Subsystem

- Responsibility:
    - Sends notifications to users about reservations, deadlines, and other important updates.
- Key Features:

- Notify users about reservation confirmations.
- Send reminders for overdue books or upcoming deadlines.
- Notify users about new books or available seats.

6. Wishlist Management Subsystem

- Responsibility:
  - Allows users to create and manage Wishlist's for books they are interested in.
- Key Features:
  - Add or remove books from a Wishlist.
  - Notify users when a book on their Wishlist becomes available

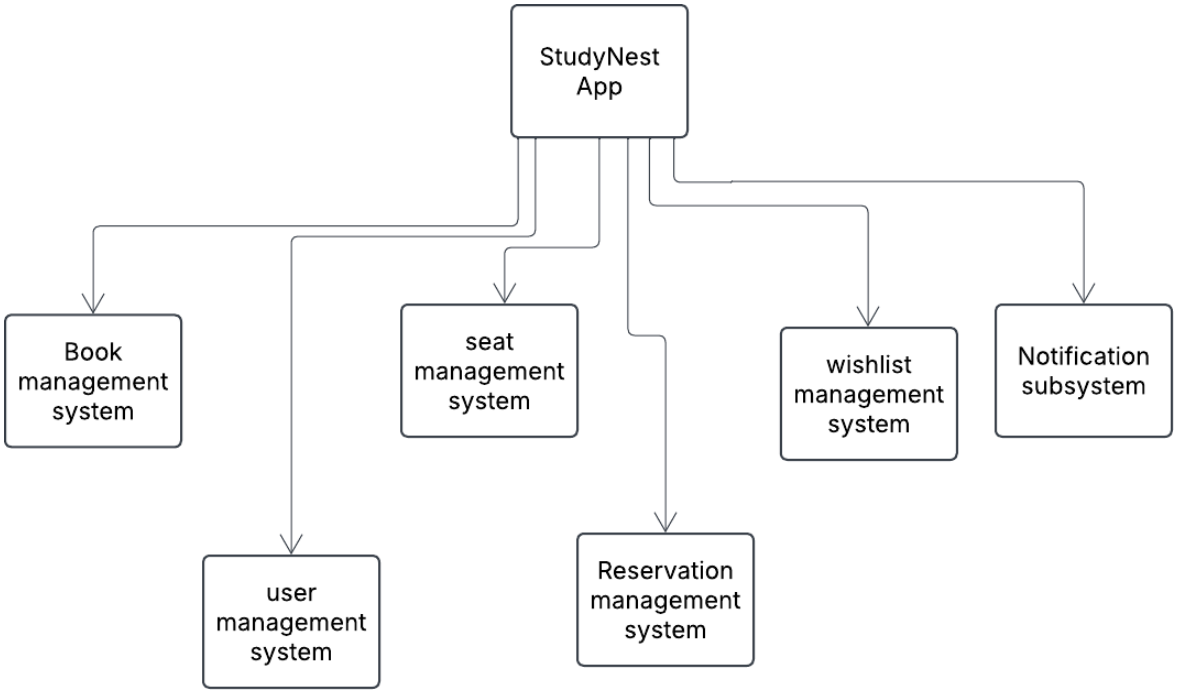


Figure 12: Subsystem decomposition diagram

3.5.2. Component diagram

1. Web Application Component

- Responsibility:
  - Represents the primary interface accessible to users for interacting with the library system.
- Key Features:
  - Provides a user-friendly interface for students, library staff, and admins.
  - Handles user interactions such as logging in, searching for books, reserving seats, and managing Wishlist's.

2. User Interface Components

- Responsibility:
  - Includes various components for user interactions.
- Key Components:
  - Registration and Login Interface:
    - Allows users to register, log in, and manage their profiles.
  - Book Search Interface:
    - Enables users to search for books by title, author, or category.
  - Seat Reservation Interface:
    - Allows students to reserve library seats.
  - Wishlist Interface:
    - Enables users to add or remove books from their wishlists.
  - Notification Interface:
    - Displays notifications for reservations, deadlines, and updates.#

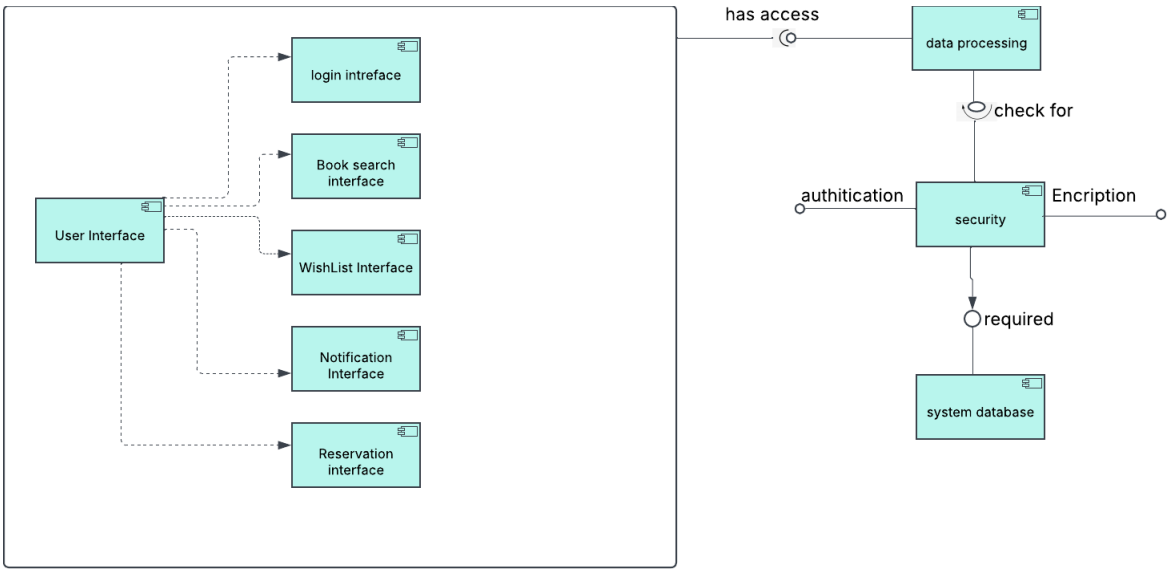


Figure 13: Component diagram

3.5.2. Persistent modeling

Ensures that data is securely stored and retrieved, maintaining data integrity and availability. The database management component handles persistent data storage, retrieval, and updates

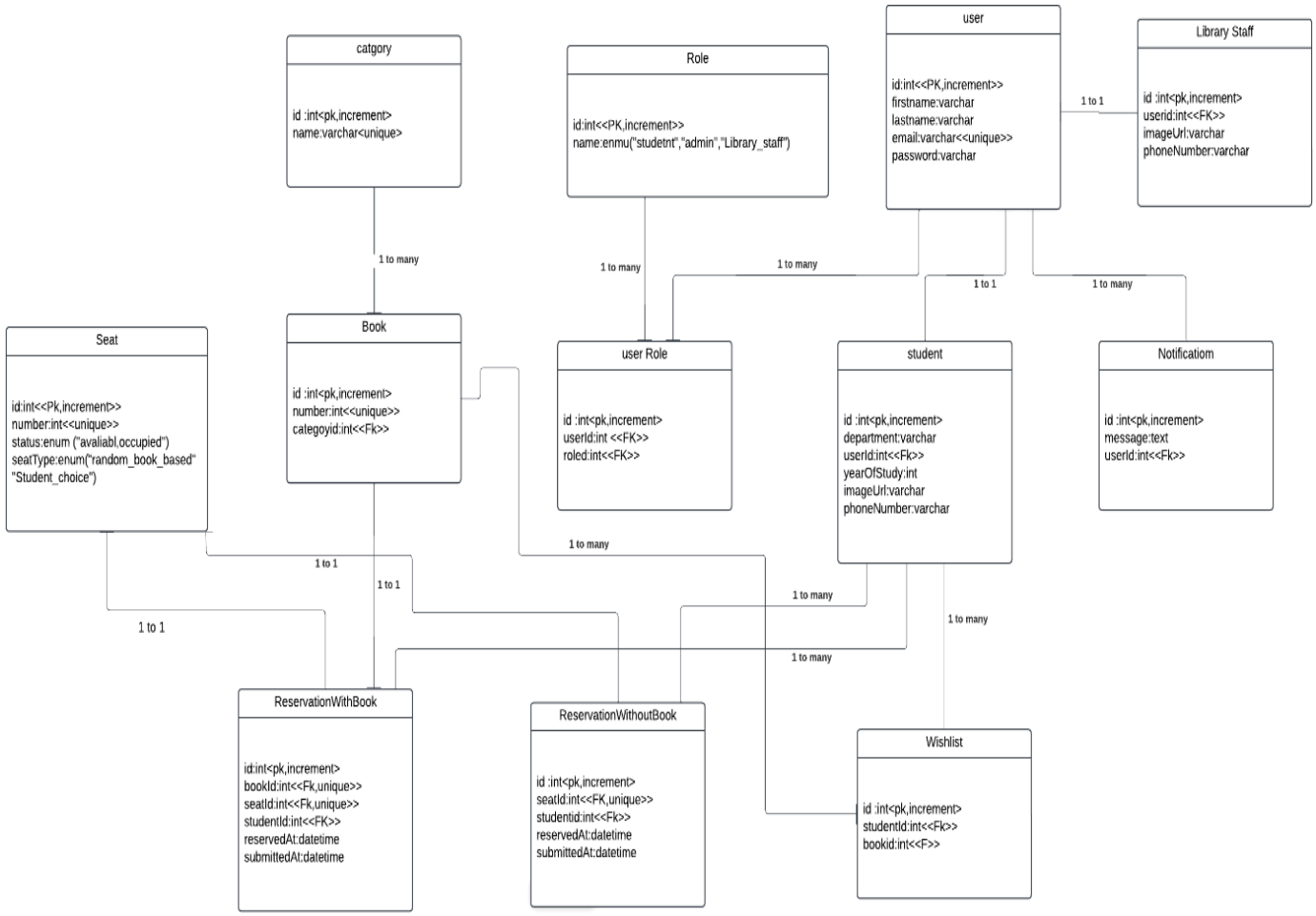


Figure 14: persistent data management

3.5.3. Access control and security

The access control component manages user authentication and authorization, ensuring that only authorized users can access specific parts of the Application or website. It also handles security measures, such as encryption of sensitive data and protection against common security threats.

3.5.4. Global software control

Coordinates the flow of control and data between subsystems, ensuring seamless communication and efficient system operation.

3.5.5. Boundary conditions

Defines the limitations and constraints under which the system operates, considering factors such as system scalability, response time, and user load.

3.6. User Interface Design

1. Homepage

- Purpose:
  - Serve as the entry point, providing users with a snapshot of available books, featured categories, and quick access to essential functions.
- Components:
  - Featured Books Carousel: Displays popular or newly added books.
  - Search Bar: Allows users to search for books by title, author, or category.
  - Navigation Menu: Provides quick access to key sections (e.g., Books, Seats, Wishlist, Notifications).

2. Book Listing Page

- Purpose:
  - Display detailed information about books available in the library.
- Components:
  - Book Images: Display cover images of books.
  - Book Details: Include title, author, category, and availability status.
  - Reserve Button: Allows students to reserve a book.

3. Registration and Login Forms

- Purpose:
  - Facilitate user onboarding and secure access to the system.
- Components:
  - Input Fields: For user details (e.g., name, email, password).
  - Secure Authentication: Includes password validation and CAPTCHA for security.

4. Admin Dashboard

- Purpose:
  - Provide tools and insights for system management, monitoring, and overseeing user and book activities.
- Components:
  - User Management Section:
    - Allows the admin to view and manage user profiles, including user details, registration status, and activity history.
  - Book Management Section:
    - Displays a list of all books in the library, enabling the admin to add, update, or remove books.
  - Seat Management Section:
    - Allows the admin to monitor and manage seat reservations.

5. Student Dashboard

- Purpose:
  - Provide students with tools for efficient book and seat management.
- Components:
  - Book Reservation Section:
    - Allows students to reserve books and view their reservation history.
  - Seat Reservation Section:
    - Allows students to reserve library seats and view seat availability.
  - Wishlist Section:
    - Allows students to add or remove books from their wishlist and receive notifications when books become available.

6. Library Staff Dashboard

- Purpose:
  - Provide library staff with tools for managing book inventory and seat reservations.
- Components:

- Book Management Section:
    - Allows staff to add, update, or remove books from the library catalog.
- Seat Management Section:
    - Allows staff to monitor and manage seat reservations.
- Inspection Section:
    - Allows staff to inspect and update the condition of books

3.7 Database Design

Database design is a critical aspect of the car rental and sales system, shaping the storage and retrieval of data to support the platform's functionality. Below is a detailed specification of the database, including the table structure and relationships.

User Table

Attribute	**Data Type**	**Relationship**
User_ID	Varchar (20)	Primary Key
First_Name	Varchar (20)	-
Last_Name	Varchar (20)	-
Email	Varchar (20)	Unique
Password_Hash	Varchar (20)	-
Registration_Date	Varchar (20)	-

Table 4: User table

Attribute	**Data Type**	**Relationship**
Student_ID	Varchar (20)	Primary Key
User_ID	Varchar (20)	Foreign Key (User)
Department	Varchar (20)	-
Year_Of_Study	Int	-
Image_URL	Varchar (20)	-
Phone_Number Varchar (20)	Varchar (20)	-

Table 5: Student table

Attribute	Data Type	Relationship
Staff_ID	Varchar (20)	Primary Key
User_ID	Varchar (20)	Foreign Key (User)
Image_URL	Varchar (20)	-
Phone_Number	Varchar (20)	-

Table 6: Staff table

Attribute	**Data Type**	**Relationship**
Role_ID	Varchar (20)	Primary Key
Name	Enum ("STUDENT", "ADMIN", "LIBRARY_STAFF")	-

Table 7: Role Table

Attribute	**Data Type**	**Relationship**
UserRole_ID	Varchar (20)	Primary Key
User_ID	Varchar (20)	Foreign Key (User)
Role_ID	Varchar (20)	Foreign Key (Role)

Table 8: User Role Table

Attribute	**Data Type**	**Relationship**
Seat_ID	Varchar (20)	Primary Key
Number	Int	Unique
Status	Enum ("available", "occupied")	-
Seat_Type	Enum ("RANDOM_BOOK_BASED", "STUDENT_CHOICE")	-

Table 9: Seat Table

Attribute	Data Type	Relationship
Book_ID	Varchar (20)	Primary Key
Number	Int	Unique
Category_ID	Varchar (20)	Foreign Key (Category)
Status	Enum ("available", "occupied")	

Table 10: Book Table

Attribute	Data Type	Relationship
Reservation_ID	Varchar (20)	Primary Key
Book_ID	Varchar (20)	Foreign Key (Book), Unique
Seat_ID	Varchar (20)	Foreign Key (Seat), Unique
Student_ID	Varchar (20)	Foreign Key (Student)
Reserved_At	Datetime	-
Submitted_At	Datetime	-

Table 11: Reservation with Book

Attribute	Data Type	Relationship
Reservation_ID		Primary Key
Seat_ID		Foreign Key (Seat), Unique
Student_ID		Foreign Key (Student)
Reserved_At		-
Submitted_At		-

Table 12: Reservation Without Book

Attribute	**Data Type**	**Relationship**
Wishlist_ID	Varchar (20)	Primary Key
Student_ID	Varchar (20)	Foreign Key (Student)
Book_ID	Varchar (20)	Foreign Key (Book)

Table 13: Wishlist Table

Attribute	**Data Type**	**Relationship**
Notification_ID	Varchar (20)	Primary Key
Message	Text	
User_ID	Varchar (20)	Foreign Key (User)

Table 14: Notification Table

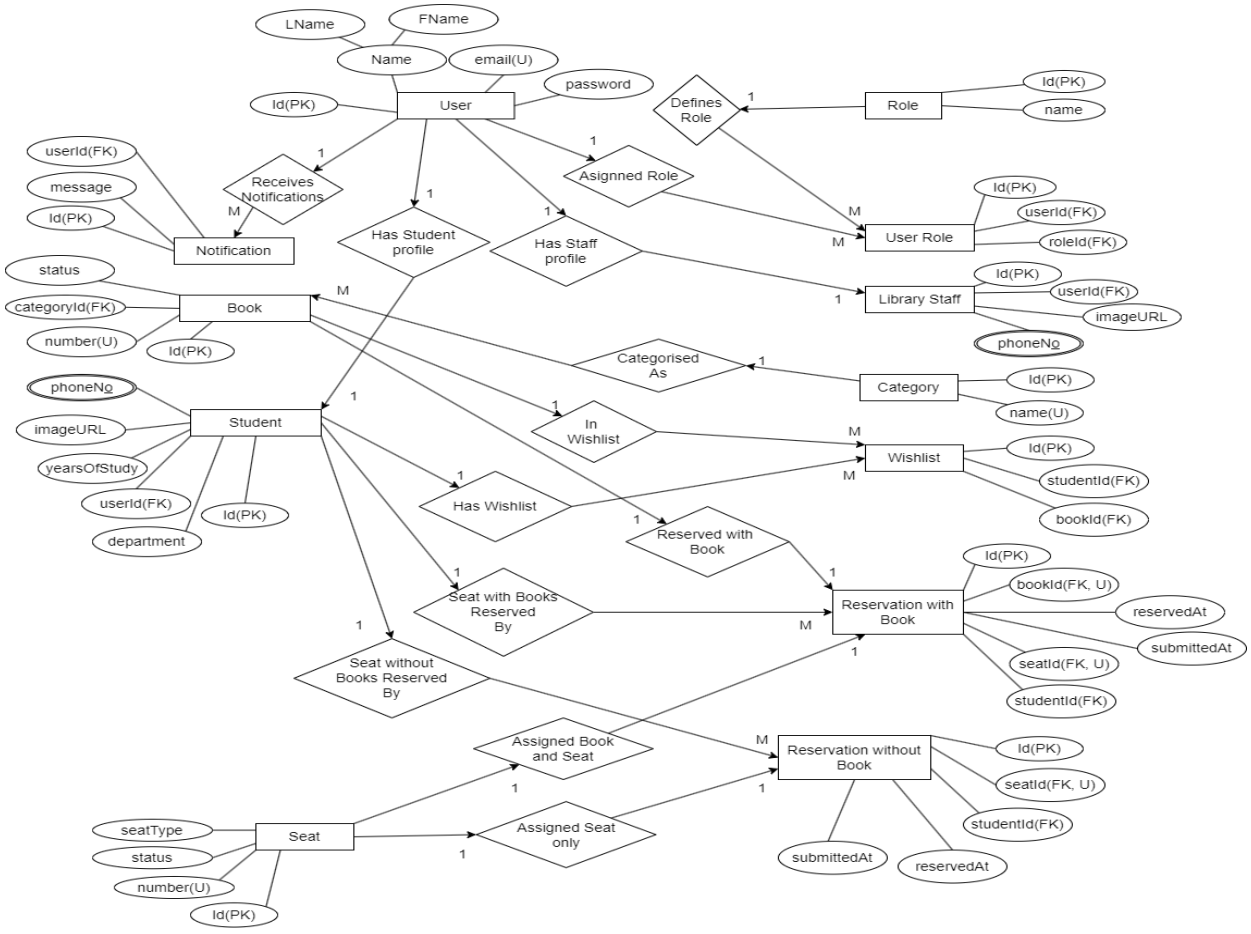


Figure 15: ER diagram

Chapter Four: Implementation and Testing

4.1 *Introduction*

This chapter delves into the implementation and testing phase of our StudyNest Library Application. Following the proposed solution outlined in earlier chapters, it details the processes involved in building the web dashboard and mobile app, integrating modules like seat management, book rental, study tracking, and community hub. We also explore the testing strategies employed to ensure efficiency and reliability.

4.2 *Overview of the Programming Language Used*

Backend (API):

The backend of this project is developed using **Node.js** and **Express.js** offering a powerful combination for building fast, scalable server-side applications using JavaScript.

Core Technologies and Libraries

Node.js: Provides a JavaScript runtime environment for building high-performance server-side applications.

Express.js: A flexible web framework used for handling routing, middleware, and API endpoints.

Backend Packages and Utilities Used:

- **Mongoose**: An Object Data Modeling (ODM) library for MongoDB, used to define schemas and interact with the database in a structured way
- **JsonWebToken (**jwt**)**: Used for creating and verifying JSON Web Tokens to implement secure authentication and session management.
- **bcrypt**: A password hashing library that ensures user passwords are securely encrypted before storage.**cookie-parser**: Parses cookies in incoming requests, enabling cookie-based authentication and session tracking.
- **cors**: Middleware that handles Cross-Origin Resource Sharing, allowing the frontend and backend to communicate across different domains.
- **nodemailer**: Used to send automated emails such as registration confirmations, password reset links, and notifications.
- **path**: A core Node.js module for handling and transforming file and directory paths.

- **multer**: Middleware used for handling multipart/form-data, making it possible to upload files (such as images) through the API.
- const upload = multer(); is used to initialize and configure the file upload middleware.
- **Agenda**: A job scheduling library used for managing background tasks (e.g., email notifications, report generation) via a modular setup (module.exports = (agenda) => {...}).

Frontend (Dashboard):

- React: A popular JavaScript library for building user interfaces (UI). It allows you to create reusable components and manage their state effectively.
- Axios: A library used for making HTTP requests from the frontend to your backend API, allowing the dashboard to interact with your backend services.
- Material-UI: A UI library built on top of React that provides pre-built components and styles based on Google's Material Design guidelines.
- Tailwind CSS: A utility-first CSS framework that provides a low-level approach to styling our application's UI.
- TypeScript: A superset of JavaScript that adds optional static typing, improving code
- readability, maintainability, and catching potential errors early in development.

Mobile App (Android and iOS):

The mobile application is developed using **React Native**, a powerful framework for building cross-platform mobile apps using JavaScript and React. This enables a shared codebase for both Android and iOS platforms, ensuring consistency and efficiency in development.

Key Technologies and Libraries Used:

- **React Native**: Allows for the development of native mobile applications using JavaScript and React. It provides a seamless user experience across both Android and iOS platforms.
- **Expo**: A framework and platform built around React Native that simplifies development, testing, and deployment. It offers access to device APIs like camera, media library, and more without needing native code.
- **expo-image-picker**: A module that enables users to select images from their device’s gallery or take new photos using the camera. It enhances interactivity and user personalization in the app.

- **AsyncStorage**: A local key-value storage system used for storing user data (e.g., login tokens, preferences) persistently on the device.
- **Formik**: A library for managing form state, validation, and submission in a clean and declarative way.
- **Yup**: A schema builder for value parsing and validation, used in combination with Formik to define and enforce form input rules.
- **react-i18next**: An internationalization framework used to support multiple languages in the app, enabling localization and accessibility for diverse user groups.

App Features and Design:

The app supports **form handling** with validation using **Formik** and **Yup**, improving input quality and user experience.

Image selection and upload are made possible through the **ImagePicker** module, facilitating user profile customization or document submission.

Multilingual support is implemented via **react-i18next**, ensuring the app can adapt its interface text based on the user's preferred language.

Data such as authentication tokens or preferences is securely stored using **AsyncStorage**, providing a smooth and persistent user experience.

4.3 *Sample User Interfaces*

Below are sample screenshots and code snippets demonstrating key UI pages and module implementations.

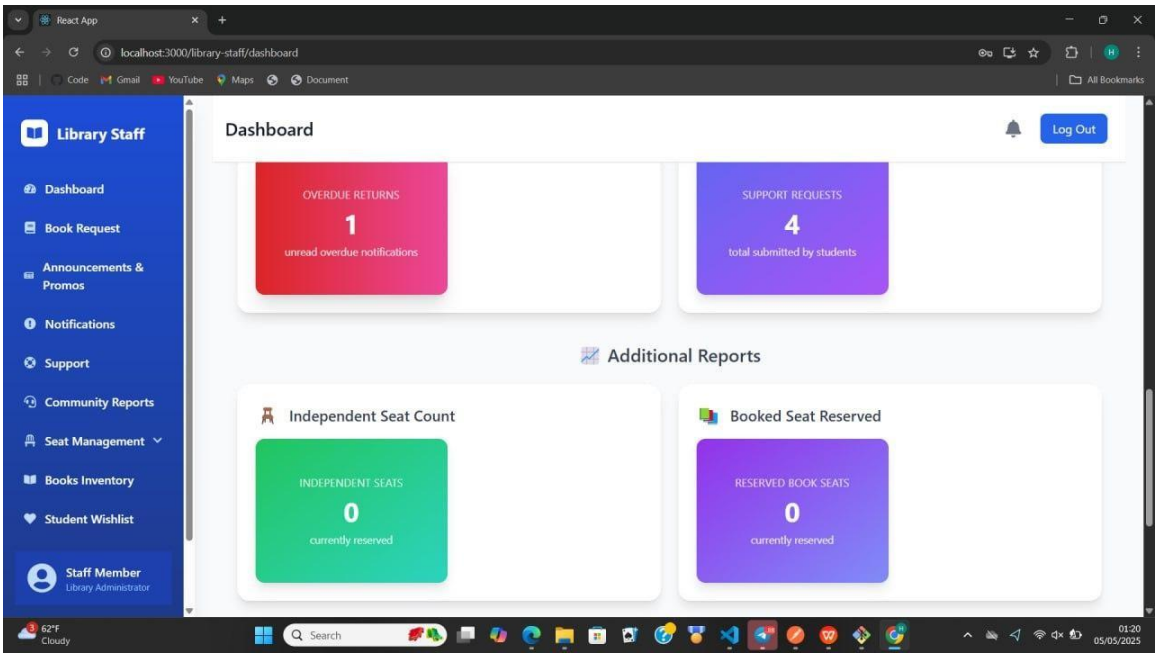


Figure 16: Library staff dashboard 1

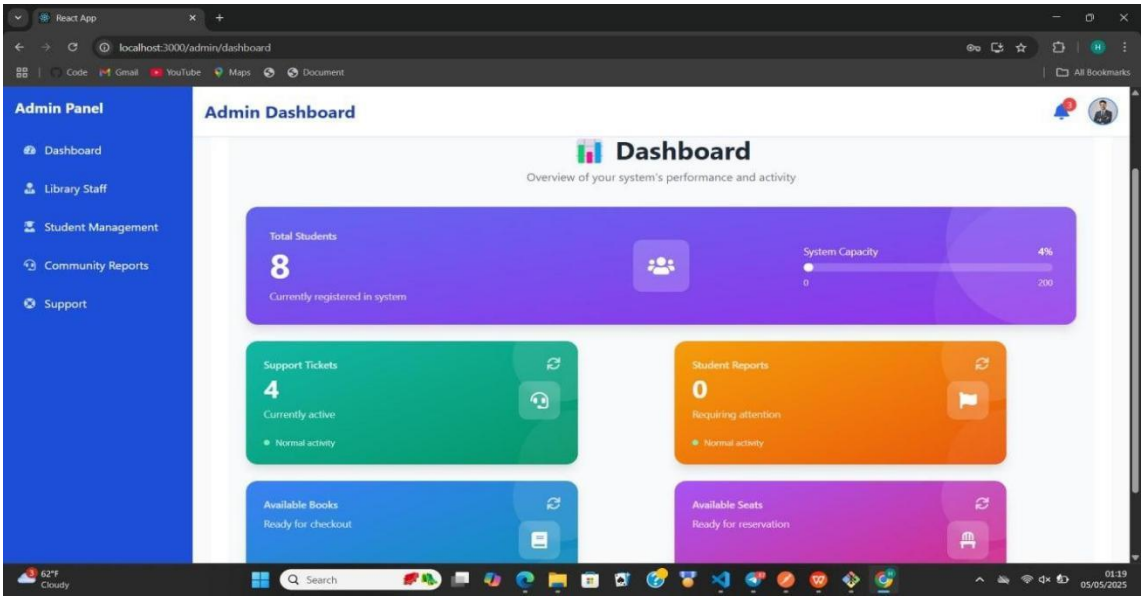


Figure 17: Admin panel (Dashboard)

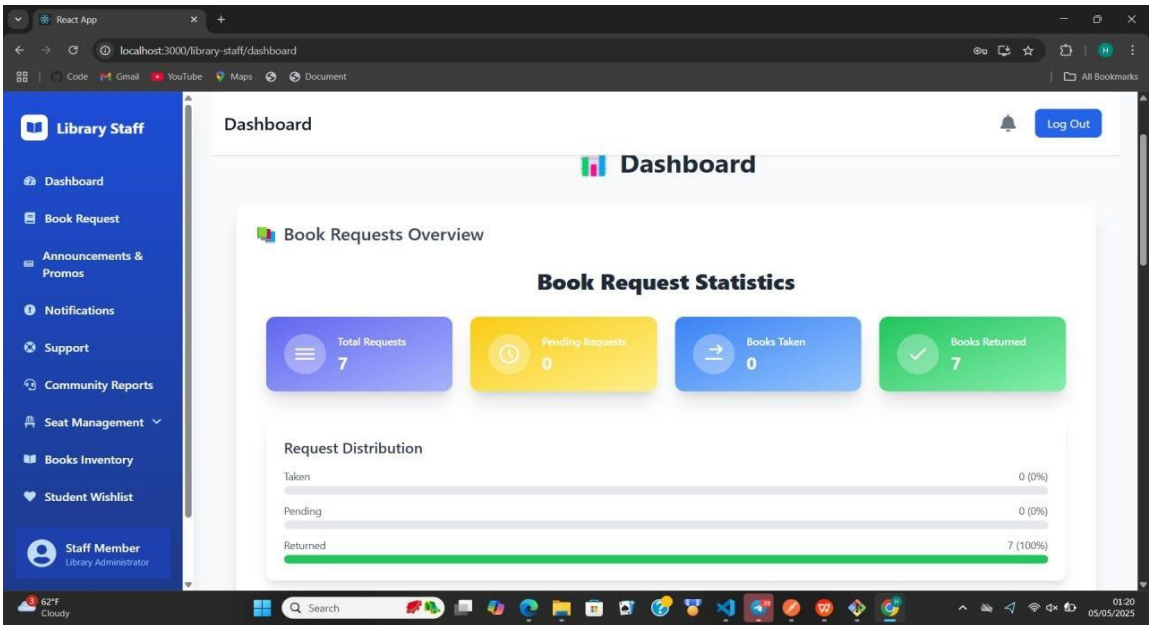


Figure 18: Library staff dashboard 2

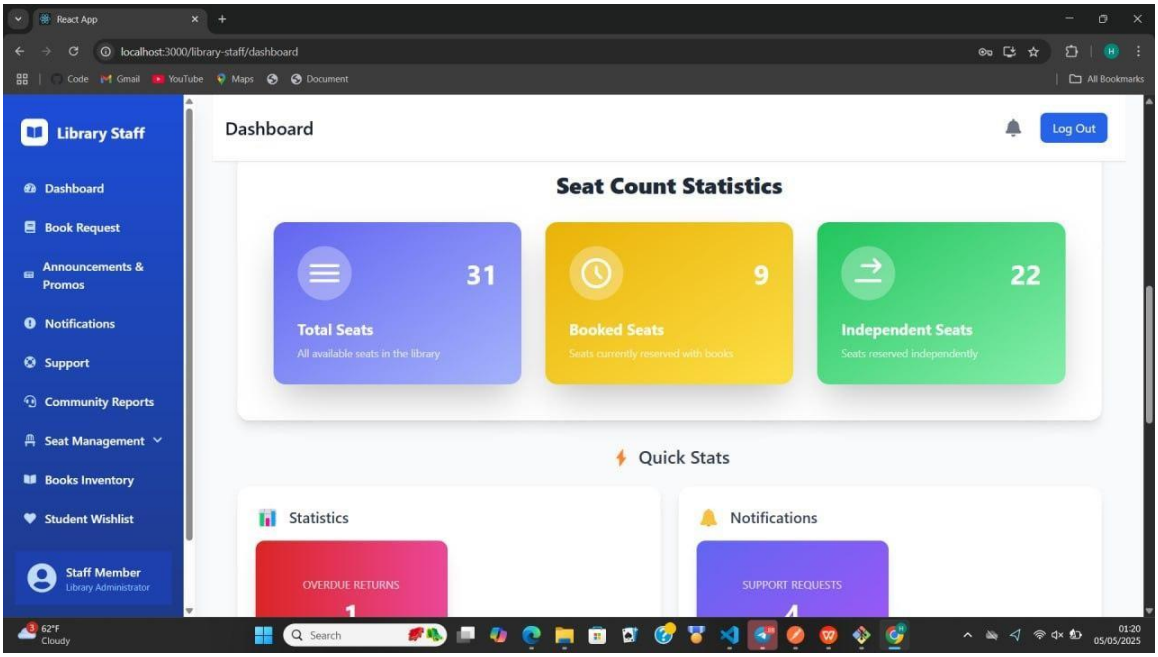


Figure 19: Library staff dashboard 3

4.4 *Application Testing*

4.4.1 Unit Testing

The goal of unit testing is to verify that each individual component of the StudyNest application behaves correctly in isolation. This early-stage testing catches logic errors before they propagate into larger integration or system tests. Below is a detailed breakdown of our unit-testing approach

4.4.1.2 Scope of Unit Testing:

- ✓ User interface components (UI)
- ✓ Data storage and retrieval functionalities
- ✓ Back-end API interaction
- ✓ Notification system

Unit testing was employed to ensure the functionality of individual components within the application. Here's a breakdown of the testing strategies used:

- ***Backend (API):*** The Mocha testing framework was utilized to create various test cases that validate the behavior of individual functions, classes, and modules within the backend code. Additionally, different API routes were tested using tools like Postman to verify the server's response to various requests.
- ***React Dashboard*:** Jest and React Testing Library (RTL) were used to prepare test cases that assess the functionality of individual components within the React dashboard. This ensures each component behaves as expected under different scenarios.
- **React Native Mobile App**: The Jest + RTL test package facilitated writing test cases to evaluate the behavior of different components in the React Native mobile app codebase.

Test Cases:

UI Component Testing:

We Verified that the user interface elements are displayed correctly and are responsive to user interactions and navigation between different screens/pages within the mobile app and Dashboard. We Ensued that UI components are aligned properly and adhere to design guidelines. sample of test are below for password field:

Test Cases for Password filed	Yeas	No	Remark
Is the object enabled?	**Y**		
Is the object encrypted?	**Y**		
Are default value provided in field, where appropriate?		**N**	
Does the field show warning for incorrect user input?	**Y**		
Does the field give warning for the user if it reaches maximum input trial?	**Y**		
Does the field have maximum amount of input characters?		**N**	
Does the field support lower- and upper-case differences?	**Y**		
Does it give hint?	**Y**		

Table 15: Test result

Automated Testing Operation using Mocha for Database interfacing:

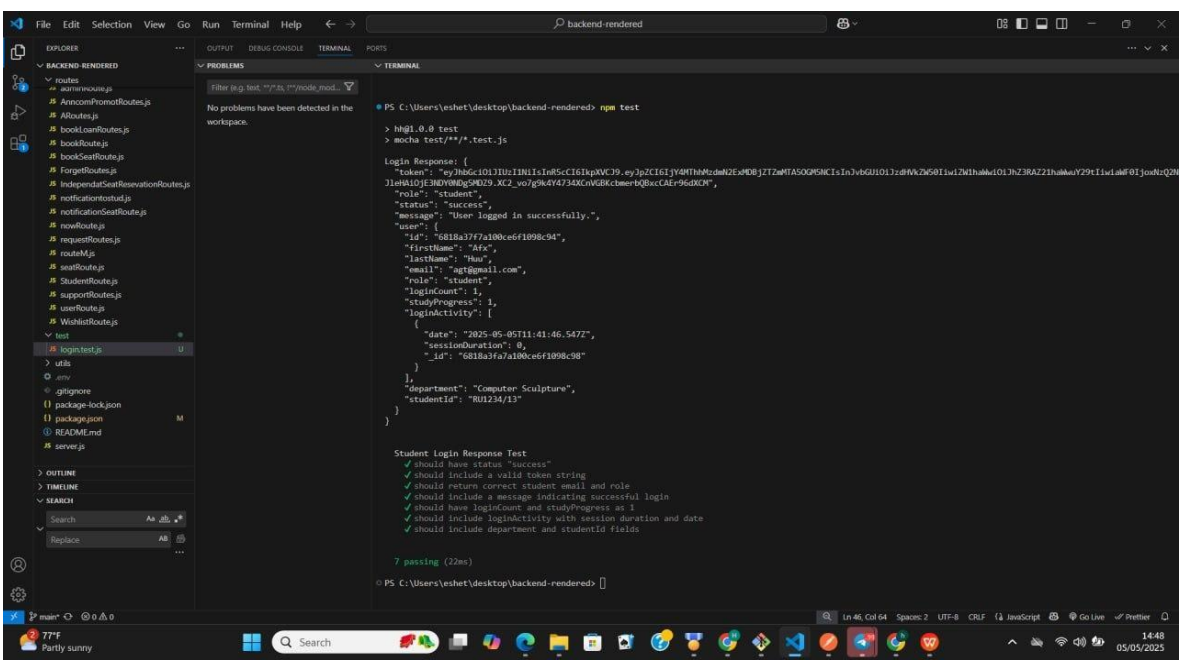


Figure 20: Backend test (Mocha test)

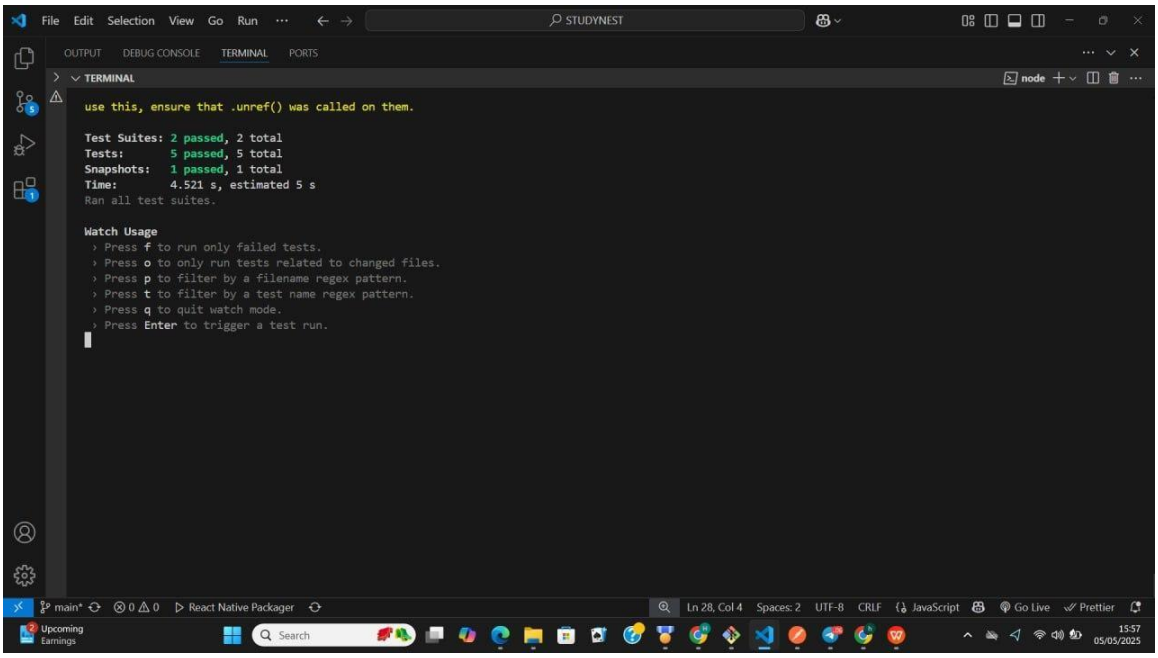


Figure 21: Jest test

4.4.1.1 Objectives

- **Correctness:** Ensure each function, class, and module returns the expected output given known inputs.
- **Robustness:** Verify error-handling paths (e.g. invalid form data, network failures) behave predictably.
- **Regression Guarding:** Quickly catch regressions when refactoring existing code.
- **Documentation:** Provide living examples of expected behavior for future developers.

4.4.3 Testing Objectives

- Verify core workflows (Authentication and Authorization, Notification) function correctly.
- Ensure real-time updates and notifications propagate reliably.
- Assess performance under normal and peak loads.
- Validate security measures: authentication, authorization, and data protection.
- Confirm usability and accessibility across devices and screen sizes.
- Ensure compatibility between web and mobile platforms.

4.4.4 Test Procedures

- Set up test environment: clone repo, install dependencies, configure environment variables.
- Define test scenarios based on functional requirements.

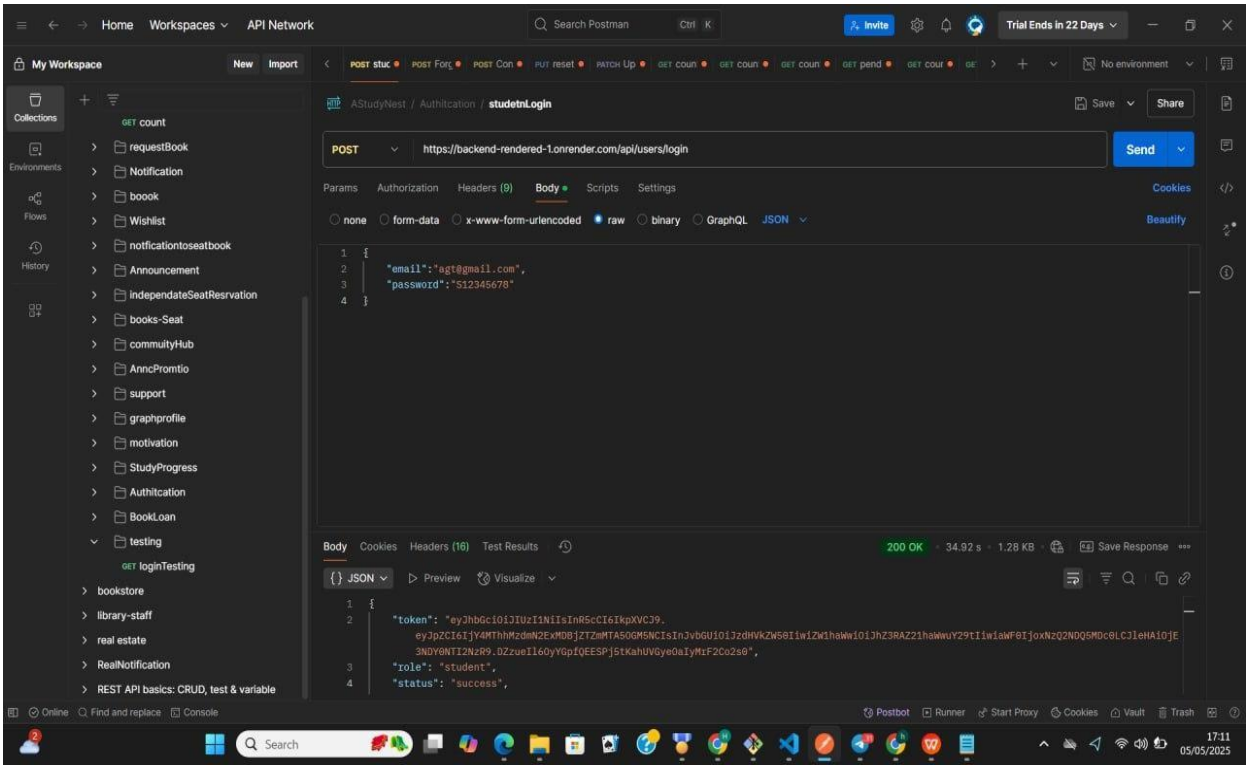
- Prepare test data and seed databases.
- Execute test cases, record results, and log defects.
- Re-run tests after fixes to ensure stability (regression testing).

Coverage targets:

Component	Framework
Backend Routes	Mocha + SuperTest
Mobile UI	RN Testing Library

4.5 *Integration Testing*

Integration testing in **StudyNest** ensures that different modules—such as user authentication, seat reservation, study tracking, and book management—work seamlessly together. For backend API integration, **Postman** is used to validate routes like login, seat status updates, and book transactions.

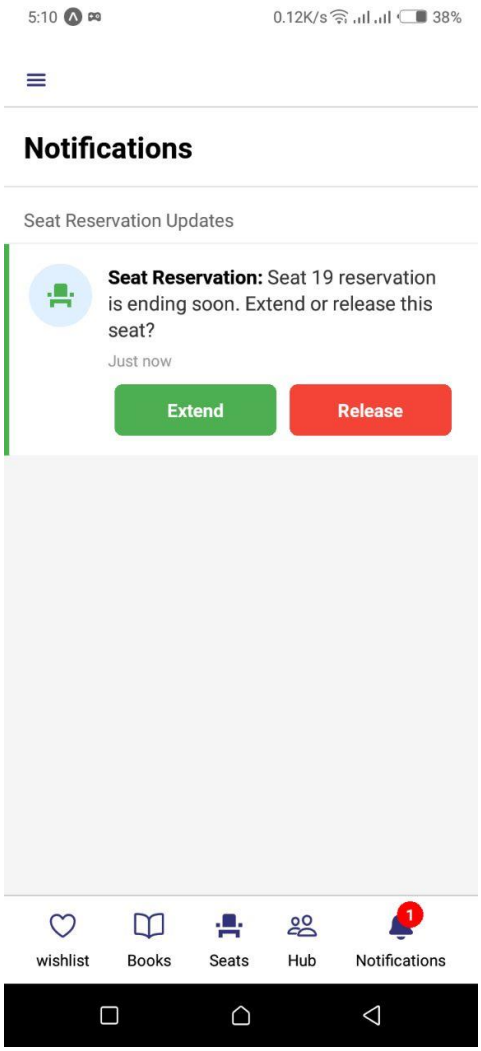


4.6 *System Testing*

System testing evaluates the complete application workflow: login, seat booking, book rental, study logging, chat, and notifications, under simulated real-world scenarios.

4.7 *Notification system testing*

Verify the notifications are triggered correctly based on the seat reservation expiration time and ensure users receive timely alerts before their reservation ends.



4.8 *User manual preparation*

4.8.1 Introduction

Welcome to the StudyNest Library Application User Manual. This guide is intended for three audiences—students, library staff, and administrators—and explains how to install, navigate, and use both our web dashboard and mobile app.

By following this manual, you'll learn how to reserve seats, borrow and return books, track study sessions, participate in department chats, and manage your account securely.

4.8.2 System Requirements

- **Web Dashboard**
  - Supported browsers: Chrome v90+, Firefox v85+, Safari v14+
  - Screen resolution ≥ 1280 × 720
- **Mobile App**
  - Android 8.0+ or iOS 13+
  - ≥ 50 MB free storage
  - Internet connectivity for full functionality

4.8.2.1 Installation & Access

- **Student Portal (Public)**
  1. Mobile: Download "StudyNest" from Google Play or the App Store.
- **Staff & Admin Dashboard (Private)**
  1. Connect to campus network (Wi-Fi or VPN).
  2. In browser, navigate to https://admin.studynest.edu.

4.8.2.2 Account Registration & Activation

- **Students**
  1. On the public site, click **Register**.
  2. Enter institutional email and create a password.
  3. Receive verification link by email; verify to set status to **Pending Approval**.
  4. Once approved by an administrator or staff, you'll get an **Activation** email—complete MFA setup.
- **Library Staff**
  1. Staff accounts are created by an Administrator.
  2. Staff receive an invite link to set their password and enroll in MFA.
- **Administrators**
  - One time "Super-Admin" account is provisioned at system launch. No self-registration.

4.8.3 Navigation & UI Overview

4.8.3.1 Student Portal

- **Header**: Global search, notifications bell, user avatar.
- **Navbar**:

- **Wishlist** | **Books** | **Seat** | **Hub** | **Notification**

4.8.3.2 Staff Dashboard

- Accessible **only** on campus network or VPN.
- **Sidebar**:
- **Notification** | **Seat Oversight** | **Book Inventory** | **Announcements** | **Reports** | **Support** | **Book Request**

4.7.3.2 Admin Pannel

- Accessible **only** on campus network or VPN.
- **Sidebar**:
  - **Library Staff** | **Student Management** | **Community Reports** | **Reports** | **Support**
- **Top-bar**: Quick-links to **Create Staff**, **Approve Students**, **System Settings**

4.7.4. Core Workflows

4.7.4.1 Seat Management

- **Students**
  1. Navigate to **Seats**.
  2. Observe the floor from ceiling point of view then press continue
  3. View live availability, select a seat, click **Reserve**.
  4. To release, click **My Reservations → Release**.
- **Staff**
  1. Under **Seat Oversight**, view all active reservations.
  2. Manually release or reassign seats if needed.

4.7.4.2 Book Rental

- **Students**
  1. Search in **Books**, click **Borrow** or **Return**.
- **Staff**
  1. Confirm check-out/check-in under **Book Inventory**.
  2. Update status (lost, damaged) where applicable.

4.7.4.3 Study Tracking

- **Students**
  1. Open **Study**, click **Start Session**; click **End Session** when done.
  2. View dashboards under **Analytics**.
- **Admins/Staff**

- Access aggregate usage reports.

4.7.4.4 Community Hub

- **Students & Staff**
  1. Go to **Chat**, select your department channel.
  2. Post messages, @mention colleagues, attach files (staff only).

virtual users showed <200ms p95 response times for critical endpoints.

Chapter Five Conclusion and Recommendation

5.1 Conclusion

This project has taken a thoughtful and practical approach to solving the prevalent issue of inefficient library seat management in Ethiopian academic institutions.

Chapter One introduced the StudyNest Library Application by outlining the challenges of manual seat tracking, describing the objectives of automating the process, and identifying the key stakeholders—students, library staff, and institutional administrators. It also detailed the research methods and system development approaches chosen to guide the project's trajectory.

Chapter Two explored the current operational procedures in libraries, identifying inefficiencies and pain points that affect both users and administrators. By analyzing existing workflows and documentation, we defined a set of functional and non-functional requirements. System modeling techniques were applied to envision the improved user and administrator experience, laying the groundwork for a more efficient seat reservation and tracking system.

In Chapter Three, we transitioned into system design using object-oriented principles. Use case diagrams were employed to define the roles of various actors, while class and sequence diagrams helped structure the interactions and data flow between system components. The user interface was also prototyped to reflect the simplicity and accessibility of the application, prioritizing usability for student users.

Chapter Four documented the implementation and testing phases of the StudyNest application. Built with Angular for the frontend and integrated with a robust backend system, the application was tested extensively for functionality, responsiveness, and reliability. The testing phase confirmed the system's ability to manage seat allocation effectively while collecting valuable real-time data to support decision-making by library staff.

In conclusion, StudyNest is not just a digital tool—it's a comprehensive solution aimed at transforming academic environments by enhancing the way students interact with library spaces. By simplifying seat access, providing occupancy insights, and streamlining administrative processes, StudyNest stands to significantly improve the learning experience in Ethiopian institutions and beyond.

5.2 Recommendation

Based on the development process and insights gained throughout the StudyNest project, the following recommendations are proposed to ensure its long-term effectiveness, scalability, and user satisfaction:

i. Institutional Integration: Partnering with universities and schools for seamless integration with student ID systems and academic portals will strengthen adoption and streamline user authentication.

ii. Feature Expansion: Incorporate advanced features such as seat occupancy analytics, heatmaps for high-usage times, booking history, and integration with academic calendars for better time planning.

iii. Training and Support: Offer orientation sessions for librarians and user guides for students to ensure smooth onboarding and reduce resistance to change.

iv. Feedback and Iteration: Implement feedback collection mechanisms within the app to continuously improve usability and functionality based on real-world user experiences.

v. Scalability Considerations: Design the backend to handle concurrent users from multiple institutions and support multiple branches or campuses under a single system.

vi. Promotion and Awareness: Conduct promotional campaigns, student ambassador programs, or demo sessions to raise awareness about the app's benefits and encourage early adoption.

vii. Sustainability Planning: Explore monetization options or institutional licensing models to support long-term maintenance, server costs, and future updates.

viii. Impact Evaluation: Develop key performance indicators (KPIs) to measure the app's effect on study efficiency, seat utilization, and administrative workload reduction, and conduct periodic assessments to refine system goals.

By implementing these recommendations, StudyNest can become a scalable and impactful digital solution for academic institutions, promoting efficiency, enhancing student experiences, and contributing to a more organized and tech-enabled educational environment.

5.3 Summary

The **StudyNest** platform represents an innovative solution designed to transform how university students in Ethiopia engage with academic resources, optimize study time, and connect with peers. This integrated system—comprising a student-focused mobile app and a secure web-based dashboard for administrators and library staff—addresses the critical need for efficient library interaction, digital study planning, and community-driven academic support.

By moving beyond traditional paper-based or inefficient library systems, **StudyNest** digitizes key processes like seat reservations, study time tracking, and academic content sharing. Students can easily reserve study spots, receive real-time updates on library capacity, and access personalized study analytics. The mobile app fosters self-discipline and productivity while also encouraging collaboration through department-based chat rooms, enabling students to share resources and seek peer guidance.

The platform also includes features specifically designed for library staff and administrators. Only staff authorized by the system admin—who is the sole entity with privileges to add or manage staff access—can utilize the web dashboard. This setup ensures a secure and structured flow of information. Administrative access can be restricted to institutional networks, protecting sensitive data and limiting misuse.

During the development lifecycle, the team emphasized structured system design, including modular

architecture, component diagrams, secure deployment practices, and intuitive user interfaces. This ensures a seamless and practical experience for all stakeholders—from students and staff to system administrators.

The impact of **StudyNest** goes beyond digital efficiency; it promotes academic accountability, resource optimization, and student well-being. With its scalable architecture and focus on accessibility, **StudyNest** serves as a potential model for higher education institutions across Ethiopia and other developing regions seeking to modernize academic support systems and foster student success in a digital era.

5.4 References

[1]. Kaur A., and Mahajan P., Automation and Networking of Libraries: A Study of Government Colleges in Punjab, International Journal of Library and Information Studies, 2016. 6(3): p. 1–12. [Google Scholar]

[2]. Tripathi M., and Jeevan V.K.J., Library Automation and Networking in India: Problems and Prospects, Library Review, 2010. 59(3): p. 180–191. doi: 10.1108/00242531011031112 [CrossRef] [Google Scholar]

[3]. Rehman S.U., and Shafique F., Use of Academic Libraries: A Survey of the Students of Pakistan, Chinese Librarianship: An International Electronic Journal, 2011. 31. [Google Scholar]

[4]. Adebayo O., Abdulwahab O., and Okorie F., An Online Library Seat Reservation System, Journal of Multidisciplinary Engineering Science and Technology (JMEST), 2014. 1(5): p. 72–78. [Google Scholar]

[5]. IFLA, Global Vision Report Summary, International Federation of Library Associations and Institutions (IFLA), 2018. Available at: https://www.ifla.org/publications/node/11905 [Google Scholar]

[6]. Bates M.J., Toward an Integrated Model of Information Seeking and Searching, The New Review of Information Behaviour Research, 2002. 3: p. 1–15. [Google Scholar]

[7]. Odu N.E., and Aghadiuno P., Digital Transformation in Academic Libraries in Nigeria: Challenges and Opportunities, Library Philosophy and Practice (e-journal), 2021. 5672. [Google Scholar]

[8]. Zhou Y., and Wang C., Design and Implementation of a Smart Library Seat Management System, International Journal of Emerging Technologies in Learning (iJET), 2020. 15(2): p. 68–82. doi: 10.3991/ijet.v15i02.11541 [PMC free article] [CrossRef] [Google Scholar]

5.5 Appendix

```
const mongoose = require("mongoose");
const asyncHandler = require("express-async-handler");
const BookRequest = require("../model/bookRequest");
const Book = require("../model/bookModel");
const User = require("../model/userModel");
const Seat = require("../model/seatModel");
```

```
const Wishlist = require("../model/wishlistModel");
const NotifcactionForseat = require("../model/notifactionForSeat");
const { Expo } = require('expo-server-sdk');

const Notification = require("../model/Notification");
// □ Request a Book (Students Only)
const requestBook = asyncHandler(async (req, res) => {
  const { bookId, addToWishlist } = req.body; // Accept `addToWishlist` from the request
  const studentId = res.locals.id;

  // Validate the book ID format
  if (!mongoose.Types.ObjectId.isValid(bookId)) {
    return res.status(400).json({ status: "failed", message: "Invalid book ID format" });
  }

  // Check if the book exists
  const book = await Book.findById(bookId);
  if (!book) {
    return res.status(404).json({ status: "failed", message: "Book not found" });
  }

  // Check if the student already has a pending request for this book
  const existingRequest = await BookRequest.findOne({
    student: studentId,
    book: bookId,
    status: "pending",
  });

  if (existingRequest) {
    return res.status(400).json({
      status: "failed",
      message: "You already have a pending request for this book.",
    });
  }
```

```
if (book.availableCopies > 0) {
  // Create a pending request for the book
  const request = await BookRequest.create({
    student: studentId,
    book: bookId,
    status: "pending", // Initial status is always "pending"
    takenAt: null, // No book has been taken yet
  });

  return res.status(200).json({
    status: "success",
    message: "Book request submitted successfully. Awaiting approval.",
    data: request,
  });
} else {
  // Book is unavailable
  if (addToWishlist === true) {
    // Add the student to the wishlist if they explicitly requested
    const existingWishlist = await Wishlist.findOne({ student: studentId, book: bookId });
    if (!existingWishlist) {
      await Wishlist.create({ student: studentId, book: bookId });
    }

    return res.status(200).json({
      status: "waiting",
      message: "Book is currently unavailable. You have been added to the wishlist.",
    });
  } else if (addToWishlist === false) {
    // Student does not want to join the wishlist
    return res.status(200).json({
      status: "unavailable",
      message: "Book is currently unavailable. You chose not to join the wishlist.",
    });
  } else {
    // Prompt the student to decide whether to join the wishlist
```

```
      return res.status(200).json({
        status: "unavailable",
        message: "Book is currently unavailable. Would you like to be added to the wishlist?",
      });
    }
  }
});

// □  Approve Book Request (Library Staff)




// □  Approve Book Request (Library Staff)



const approveBookRequest = asyncHandler(async (req, res) => {
  const { requestId } = req.params;
  const staffId = res.locals.id; // Extract staff ID from the token

  // Validate request ID
  if (!mongoose.Types.ObjectId.isValid(requestId)) {
    return res.status(400).json({ status: "failed", message: "Invalid request ID format" });
  }

  // Check if the user is authorized (library-staff)
  const staff = await User.findById(staffId);
  if (!staff || staff.role !== "library-staff") {
    return res.status(403).json({ status: "failed", message: "Only library staff can approve requests" });
  }

  // Fetch the request and populate the associated book
  const request = await BookRequest.findById(requestId).populate("book");
  if (!request) {
    return res.status(404).json({ status: "failed", message: "Book request not found" });
  }
```

```
if (request.status !== "pending") {
  return res.status(400).json({
    status: "failed",
    message: `This request has already been processed. Current status: ${request.status}`,
  });
}

const book = request.book;

// Ensure the book has available copies before approving
if (book.availableCopies <= 0) {
  return res.status(400).json({ status: "failed", message: "No copies available" });
}

// Decrement available copies
const updatedBook = await Book.findByIdAndUpdate(
  book._id,
  { $inc: { availableCopies: -1 } }, // Decrement available copies
  { new: true }
);

if (!updatedBook) {
  return res.status(500).json({ status: "failed", message: "Failed to update book availability" });
}

// Find an available seat of type "book"
const availableSeat = await Seat.findOne({ type: "book", isAvailable: true }).sort({ seatNumber: 1 });
if (!availableSeat) {
  return res.status(400).json({ status: "failed", message: "No book-related seats available" });
}

// Assign the seat to the student
availableSeat.isAvailable = false;
availableSeat.reservedBy = request.student; // Link to the student
```

```
  availableSeat.reservedAt = new Date();
  await availableSeat.save();

  const returnDeadline = new Date(Date.now() + 2 * 60 * 1000);
  // Update the request status to 'taken' and assign the seat
  request.status = "taken";
  request.takenBy = staffId;
  request.takenAt = new Date();
  request.seat = availableSeat.seatNumber;
  request.returnDeadline = returnDeadline; // Set the deadline
  await request.save();

  // Create a notification for the student
  await NotifcactionForseat.create({
    user: request.student,
    book: book._id,
    seat: availableSeat.seatNumber,
    message: `Your book "${book.name}" has been assigned to seat ${availableSeat.seatNumber}. Please
return by ${returnDeadline.toLocaleTimeString()}.`,
    type: 'seat_assigned',
    deadline: returnDeadline
  });

  setTimeout(async () => {
    const currentRequest = await BookRequest.findById(requestId);
    if (currentRequest?.status === 'taken') {
      await NotifcactionForseat.create({
        user: request.student,
        book: book._id,
        seat: availableSeat.seatNumber,
        message: `REMINDER: Please return "${book.name}" from seat ${availableSeat.seatNumber}
soon.`,
        type: 'return_reminder'
      });
    }
```

```
  }, 1 * 60 * 1000);

  setTimeout(async () => {
    const currentRequest = await BookRequest.findById(requestId);
    if (currentRequest?.status === 'taken') {
      await NotifcactionForseat.create({
        user: request.student,
        book: book._id,
        seat: availableSeat.seatNumber,
        message: `OVERDUE: Please return "${book.name}" from seat ${availableSeat.seatNumber} immediately!`,
        type: 'return_overdue'
      });

      // Also notify staff
      await NotifcactionForseat.create({
        user: staffId, // Ensure this is a valid user ID
        book: book._id, // Ensure this is a valid book ID
        seat: availableSeat.seatNumber, // Ensure this is a valid seat number
        message: `Student ${request.student.name} has overdue book "${book.name}" at seat ${availableSeat.seatNumber}`,
        type: 'return_overdue'
      });
    }
  }, 2 * 60 * 1000);

  res.status(200).json({
    status: "success",
    message: `Book marked as taken successfully. Please return by ${returnDeadline.toLocaleTimeString()}`,
    request,
    returnDeadline
  });
});
```